

동시성을 위한 Go

4편: Go 동시성 패턴

남궁명수

[4] Go의 동시성 패턴	4
(1) 개요	4
(2) 빈 인터페이스(interface{}) 채널 사용에 대한 배경	4
(3) 패턴 학습을 위한 준비 코딩	5
[4-1] 구속	6
(1) 안전한 동시성 제어를 위한 선택지	6
(2) 구속 패턴의 개념과 임계 영역 제거	6
(3) 규칙에 의존하는 임의적 구속	6
(4) 컴파일러가 강제하는 어휘적 구속	8
(5) 구속 패턴이 가져다주는 이점과 결론	10
[4-2] for-select 루프 패턴	11
(1) 시나리오1: 순회 변수를 채널로 송신하기	11
(2) 시나리오2: 종료될 때까지 무한 반복하기	12
(3) 결론	14
[4-3] 고루틴 누수 방지	15
(1) 고루틴 누수란 무엇인가?	15
(2) 해결책 1: 데이터 수신 측 고루틴 누수 방지	16
(3) 해결책2: 데이터 송신 측 고루틴 누수 방지	17
(4) 결론	20
[4-4] or-channel	21
(1) or-channel 패턴의 개념	21
(1) or-channel 재귀 함수 소스코드 구현	21
(3) 작동 원리와 파급 효과 분석	23
(4) 결론	23
[4-5] 에러 처리(Error Handling) 패턴	24
(1) 동시성 환경에서 에러 처리의 난제	24
(2) 해결책: 결과와 에러를 하나로 묶는 관심사 분리	25
(3) 유통 에러 패턴의 실무 확장성	27
(4) 결론	28
[4-6] 파이프라인	29
(1) 파이프라인 패턴의 개념	29
(2) 절차적 함수로 구현하는 배치 처리 스테이지	30
(3) 메모리 절약을 위한 스트림 처리 스테이지	32
(4) 절차적 스트림 처리의 한계와 동시성 아키텍처의 필요성	33
(5) 결론	33
[4-7] 파이프라인을 구축하는 정석적인 모범 설계 기법	34
(1) 채널 기반 동시성 파이프라인 전체 소스코드	34
(2) 우리가 동시성 파이프라인을 통해 얻은 이점	36
(3) 데이터가 파이프라인을 타고 흐르는 연산 타임라인 분석	36
(4) 취소 신호의 도미노 파급 효과	37
(5) 결론	37
[4-8] 실무용 제너레이터 패턴	38
(1) 무한 루프 스트림 제어: repeat와 take 스테이지	38
(2) 동적 연산 제너레이터: repeatFn 스테이지	39
(3) 빈 인터페이스 채널 설계와 타입 변환 스테이지	41
(4) 범용 인터페이스 아키텍처 vs 고유 타입 아키텍처 성능 검증	43
(5) 결론	46
[4-9] 팬아웃 / 팬인(Fan-out / Fan-in)	47
(1) 팬아웃과 팬인의 성립 조건	47
(2) 순차적 파이프라인의 속도 한계 증명	48
(3) 초간단 팬아웃 구현 기법	50
(4) 다중 채널을 하나로 묶는 팬인(Fan-in) 알고리즘 구현	51

(5) 팬아웃, 팬인 결합 가속 파이프라인 전체 소스코드	52
(6) 결론	56
[4-10] tee-channel 패턴	57
(1) tee-channel 패턴의 전체 소스코드 구현	57
(2) 섀도잉과 Nil 채널을 활용한 비블로킹 분배 법칙	60
(3) 실무 설계 시 주의해야 할 결합도 특성	60
(4) 결론	61
[4-11] bridge-channel	62
(1) bridge-channel 패턴의 전체 소스코드 구현	63
(2) 이중 루프 제어를 통한 Seamless(심리스) Stream 아키텍처의 원리(경계 없는 스트림)	65
(3) 결론	66
[4-13] Queuing	67
(1) 큐를 도입해도 프로그램 전체 속도는 빨라지지 않는다.	67
(2) 대기열(Queue)의 진짜 유용성과 실무적 가치	68
(3) 효율적인 배치 처리를 증명하는 벤치마크(Chunking 성능 검증)	69
(4) 시스템 전면 마비를 부르는 ‘죽음의 나선’과 해결책	71
(5) 큐 크기 결정을 위한 리틀의 법칙(Little’s Law)	71
(6) 결론	73
[4-14] context 패키지	74
(1) context 패키지의 기본 구조와 인터페이스 스펙	74
(3) 고전적인 Done 채널 방식과의 아키텍처 비교	75
(4) 고속 실패 처리를 위한 데드락 사전 검증 기법	79
(5) 컨텍스트 가방을 통한 요청 범위 데이터 유동	82
(6) 컨텍스트 가방에 담을 올바른 데이터 판별 법칙	84
(7) 결론	85

[4] Go의 동시성 패턴

(1) 개요

우리는 지금까지 Go언어가 자랑하는 동시성의 기초 벽돌인 고루틴, 채널, 그리고 sync 패키지의 다양한 저수준 동기화 도구들을 다루는 법을 배웠다.

이번 4장에서는 이 도구들을 한 차원 높게 조립하여, 현실 세계의 대용량 서비스를 안정적이고 확장 가능하며 유지보수하기 쉽게 만들어주는 ‘고급 동시성 디자인 패턴’의 세계로 깊숙이 들어가 보겠다.

본격적인 패턴 학습에 앞서, 앞으로 등장할 예제 코드들의 독특한 형태에 대해 먼저 짚고 넘어갈 부분이 있다.

(2) 빈 인터페이스(interface{}) 채널 사용에 대한 배경

앞으로 진행될 대다수의 동시성 패턴 예제에서는 데이터를 주고받을 때 특정 타입(예: int, string) 대신 빈 인터페이스를 채우는 채널을 자주 사용하게 된다.

Go 개발자들 사이에서 이 빈 인터페이스를 남용하는 것은 타입 안정성을 해친다는 이유로 꽤 많은 논쟁이 있는 주제이다.

그럼에도 불구하고 본 장의 예제에서 이를 과감하게 선택한 이유는 두 가지 명확한 이유가 있다.

- 첫 번째: 예제의 간결함

어떤 특정한 도메인 데이터 타입에도 종속되지 않음으로써, 책 전체에서 패턴의 본질적인 핵심 흐름을 가장 짧고 간결한 코드로 보여주기 위함이다.

- 두 번째: 패턴의 본질 표현

특정 타입에 얽매이지 않고 데이터가 흘러가는 파이프라인의 순수한 아키텍처 구조 그 자체를 직관적으로 묘사하는 데 빈 인터페이스가 더 적합하기 때문이다. 이 부분은 뒤이어 나올 ‘파이프라인’ 섹션에서 더욱 깊게 깨닫게 될 것이다.

만약 이 빈 인터페이스 구조가 타입 시스템 관점에서 너무 불편하게 느껴지신다면, 실무에 적용하실 때 Go의 코드 생성기 기술이나 제네릭을 활용하여 여러분이 원하는 타입을 주입하는 방식으로 패턴을 변형해 사용하면 된다.

(3) 패턴 학습을 위한 준비 코딩

앞으로 배울 패턴들이 어떤 형태로 시작되는지, 빈 인터페이스 채널을 활용해 데이터를 안전하게 흘려보내는 가장 정석적인 채널 소유권 분리 예제 코드를 작성해본다.

생략 없는 전체 코드를 확인해보자.

```
package main

import "fmt"

func main() {
    // 동시성 패턴의 기초가 되는 제너레이터(생산자) 패턴 구현
    dataGenerator := func() <-chan interface{} {
        outputStream := make(chan interface{})

        go func() {
            defer close(outputStream)
            // 빈 인터페이스 채널을 통해 서로 다른 타입의 데이터를 안전하게 전송
            outputStream <- "Pattern study started"
            outputStream <- 2026
            outputStream <- true
        }()

        return outputStream
    }

    // 생산자가 수출한 수신 전용 채널을 소비자가 가져와 순회 연산 수행
    results := dataGenerator()
    for data := range results {
        fmt.Printf("Received data from pattern pipe: %v (Type: %T)\n", data, data)
    }
    fmt.Println("Main pipeline loop finished safely.")
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Received data from pattern pipe: Pattern study started (Type: string)
Received data from pattern pipe: 2026 (Type: int)
Received data from pattern pipe: true (Type: bool)
Main pipeline loop finished safely.
```

이처럼 빈 인터페이스 채널은 동시성 아키텍처의 뼈대를 유연하고 직관적으로 설계할 수 있도록 도와준다.

[4-1] 구속

(1) 안전한 동시성 제어를 위한 선택지

동시성 코드를 작성할 때, 여러 고루틴이 안전하게 데이터를 주고받으며 버그를 일으키지 않도록 만드는 방법은 여러가지가 있다. 우리는 이미 앞 섹션에 두 가지 방법을 배웠다.

- 메모리 공유를 위한 동기화 도구 사용: 자물쇠와 같은 `sync.Mutex` 계열의 도구 활용
- 통신을 통한 동기화 조율: Go의 핵심 도구인 채널 활용

하지만 이외에도 동시성 환경에서 별도의 자물쇠 조율 없이 구조 자체만으로 무조건 안전함을 보장하는 두 가지 강력한 선택지가 더 존재한다.

- 불변 데이터
데이터를 한 번 만들면 절대 수정할 수 없게 만드는 방식이다.
여러 고루틴이 동시에 같은 데이터를 쳐다보더라도 값을 바꿀 수 없기 때문에 데이터 오염이 원천 차단된다. 만약 값을 수정하고 싶다면 원본을 복사한 뒤 원하는 부분만 수정한 새로운 복사본을 만들어야 한다. Go 언어에서는 포인터(*) 대신 값을 직접 복사해서 전달하는 방식으로 이를 구현한다.

- 구속 데이터
데이터를 오직 단 하나의 고루틴만 접근할 수 있는 영역에 격리하여 가둬두는 방식이다.
이번 섹션에서 깊게 파고들 핵심 주제이다.

(2) 구속 패턴의 개념과 임계 영역 제거

구속은 아주 단순하면서도 강력한 아이디어이다. 특정 정보나 자원에 접근할 수 있는 권한을 오직 한 번에 하나의 고루틴에게만 부여하도록 보장하는 설계 방식이다.
이 구속이 완벽하게 성립되면, 그 동시성 프로그램은 자물쇠(Mutex)로 줄을 세우거나 채널로 데이터를 주고 받는 복잡한 동기화 과정 자체가 아예 필요 없어진다.
경쟁할 상대가 없기 때문이다.

구속 패턴을 구현하는 방법은 크게 두 가지로 나뉜다.

- 임의적 구속(Ad hoc confinement)
- 어휘적 구속(Lexical confinement)

(3) 규칙에 의존하는 임의적 구속

임의적 구속이란

구조나 컴파일러의 강제성 없이, 오직 개발자들 사이의 말뿐인 규칙이나 약속에만 의존하여 자원을 격리하는 방식이다.
팀의 규칙이나 코드베이스의 관례로 “이 변수는 이 함수 안에서만 건드리자”라고 정의하는 것이다.

하지만 프로젝트의 규모가 커지고 마감 압박이 밀려오면, 누군가는 반드시 규칙을 어기기 마련이다. 툴을 통한 강제성이 없다면 언제든지 무너질 수 있는 위험한 방식이다.
임의적 구속이 왜 위험한지 보여주는 전체 코드를 확인해보자.

```

package main

import (
    "fmt"
)

func main() {
    // 공유 자원 데이터 생성
    data := make([]int, 4)
    data[0] = 10
    data[1] = 20
    data[2] = 30
    data[3] = 40

    loopData := func(handleData chan<- int) {
        defer close(handleData)
        // 관례상 이 데이터 슬라이스는 오직 이 함수 내부에서만 순회하며 접근하기로
약속함
        for i := range data {
            handleData <- data[i]
        }
    }

    handleData := make(chan int)
    go loopData(handleData)

    for num := range handleData {
        fmt.Println(num)
    }
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

10
20
30
40

```

위의 data 슬라이스는 loopData 함수 안에서도 접근할 수 있고 main 함수 루프 안에서도 접근할 수 있는 열린 구조이다. 지금은 개발자들이 약속을 잘 지켜서 loopData 내부에서만 안전하게 썼지만, 사람이 조심하는 규칙은 코드 수정이 반복되면서 언제든 깨질 수 있다. 이를 예방하려면 컴파일러가 강제로 규칙을 지게 만드는 어휘적 구속을 써야 한다.

(4) 컴파일러가 강제하는 어휘적 구속

어휘적 구속이란

언어가 제공하는 변수의 유효 범위(스코프)를 치밀하게 활용하여, 동시성 작업들이 오직 자신에게 허용된 올바른 데이터와 도구만 바라볼 수 있도록 물리적으로 격리하는 방식이다. 애초에 코드 구조상 잘못된 행동을 하고 싶어도 할 수 없도록 가두는 기술이다.

3장에서 배웠던 단방향 채널 전송 기법이 대표적인 어휘적 구속의 예시이다.

채널의 생성과 송신 권한을 하나의 함수 내부 스코프에 완벽하게 가두어 숨겨두고, 외부에는 오직 읽기 전용 채널(<-chan)만 반환하여 수출하는 구조를 전체 코드로 확인해 보자.

```
package main

import (
    "fmt"
)

func main() {
    // 채널의 소유권을 자신 내부로 어휘적 구속을 실행한 함수 정의
    chanOwner := func() <-chan int {
        // results 채널은 오직 chanOwner 함수 안에서만 보이고 생성됩니다.
        results := make(chan int, 5)

        go func() {
            defer close(results)
            for i := 0; i <= 5; i++ {
                results <- i // 송신 권한은 이 클로저 내부에 완벽히 구속됩니다.
            }
        }()

        return results // 소비자를 위해 오직 읽기 전용 뷰만 반환합니다.
    }

    consumer := func(results <-chan int) {
        for result := range results {
            fmt.Printf("Received: %d\n", result)
        }
        fmt.Println("Done receiving!")
    }

    // 메인 고루틴은 수신 전용 채널만 넘겨받으므로 값을 절대 채널에 집어넣을 수 없습니다.
    results := chanOwner()
    consumer(results)
}
```

이 구조에서는 `result := chanOwner()`를 통해 받아온 채널에 외부 유저가 실수로 값을 밀어 넣으려고 해도, 컴파일러가 단방향 채널 규칙 위반 에러를 내며 빌드 자체를 막아버린다.

멀티스레드에 대한 안전하지 않은 자원 보호하기

채널은 원래 동시성에 안전한 도구이지만, 동시성에 전혀 안전하지 않은 일반 메모리 객체(예: bytes.Buffer)도 자물쇠 없이 이 구속 패턴으로 지킬 수 있다.

고루틴 일꾼들을 여러 개 띄우되, 공용 배열 메모리를 통째로 주지않고 각 고루틴이 건드릴 수 있는 구역을 칼로 자르듯 명확하게 쪼개서(slice) 배정하면 된다.
전체 코드를 확인해 보자.

```
package main

import (
    "bytes"
    "fmt"
    "sync"
)

func main() {
    printData := func(wg *sync.WaitGroup, data []byte) {
        defer wg.Done()

        // 각 고루틴 인스턴스는 자신만의 독립된 로컬 버퍼 메모리를 가집니다.
        var buff bytes.Buffer
        for _, b := range data {
            fmt.Fprintf(&buff, "%c", b)
        }
        fmt.Println(buff.String())
    }

    var wg sync.WaitGroup
    wg.Add(2)

    // 전체 공용 데이터 슬라이스 생성
    data := []byte("golang")

    // 첫 번째 고루틴에게는 앞의 3글자 구역만 칼로 잘라서 넘겨 가둡니다.
    go printData(&wg, data[:3])

    // 두 번째 고루틴에게는 뒤의 3글자 구역만 칼로 잘라서 넘겨 가둡니다.
    go printData(&wg, data[3:])

    wg.Wait()
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
lan
gog
(참고: 동시성 실행 타이밍에 따라 출력 순서는 gog가 먼저 나올 수도 있으며 이는 정상이다.)
```

printData 함수는 전체 data 슬라이스 변수의 원본 주소를 통째로 참조하고 있지 않다.
오직 매개변수로 넘어오는 단편적인 []byte 조각 데이터만 볼 수 있다.

메인 흐름에서 고루틴을 띄울 때 `data[:3]`과 `data[3:]`으로 영역을 완벽하게 격리해서 던져주었기 때문에, 두 고루틴은 동일한 메모리 구역을 겹쳐서 건드릴 일이 제로가 된다. 덕분에 우리는 그 어떤 `sync.Mutex` 자물쇠도 채우지 않고 메모리 버퍼 연산을 동시성 환경에서 안전하게 수행할 수 있게 되었다.

(5) 구속 패턴이 가져다주는 이점과 결론

이미 자물쇠(`sync.Mutex`)나 채널 동기화 기술이 있는데도 구속 설계를 추구해야 하는 이유는 압도적인 성능 향상과 개발자의 뇌 용량 과부하 감소에 있다.

자물쇠를 잠그고 푸는 동기화 연산은 내부적으로 운영체제와 런타임 자원을 소모하는 비용이 따른다. 구속 패턴을 통해 동시성 자원을 완벽하게 격리하면, 내 프로그램 내에서 여러 고루틴이 부딪히는 병목 지점인 임계 영역 자체가 완전히 소멸한다. 자물쇠를 관리할 필요가 없으니 데드락, 라이브락, 기아 상태 같은 동시성 버그에 대해 개발자가 머리를 싸매고 고민할 필요도 사라진다.

또한, 구속된 영역 안에서는 코드를 단순한 동기적 방식(일반 한줄짜리 순차 코드)으로 편하게 작성할 수 있기 때문에 가독성이 올라가고 코드를 유지보수하기 쉬워진다.

구조로 풀 수 있는 문제는 최대한 구속 패턴으로 먼저 풀어내고, 어쩔 수 없는 공유 영역에만 동기화 도구를 채우는 것이 좋은 설계 방식이다.

[4-2] for-select 루프 패턴

Go 언어 프로그램에서 가히 가장 많이 마주치게 될 동시성 제어 패턴인 for-select 루프 패턴에 대해 알아보겠다.

이 패턴은 이름 그대로 반복문(for)과 채널 컨트롤 타워(select)를 결합한 형태에 불과하다. 기본 구조는 다음과 같다.

```
for { // 무한 루프를 돌거나 혹은 어떤 컬렉션을 순회합니다.
    select {
        // 채널을 활용해 다양한 동시성 작업을 수행합니다.
    }
}
```

(1) 시나리오1: 순회 변수를 채널로 송신하기

배열이나 슬라이스처럼 반복하여 순회할 수 있는 연속된 데이터들을 동시성 채널의 스트림 값으로 변환하여 외부로 짜주고 싶을 때 사용하는 방식이다.

중요한 점은 데이터를 열심히 보내는 도중에 부모 프로세스로부터 “작업을 중단하라”는 종료 신호(done)가 오면 즉시 안전하게 루프를 빠져나갈 수 있도록 방어벽을 세워둔다는 것이다. 전체 코드로 확인해보자.

```
package main

import (
    "fmt"
    "sync"
)

func main() {
    done := make(chan interface{})
    stringStream := make(chan string)
    var wg sync.WaitGroup

    // 데이터를 생성하여 채널로 밀어 넣어주는 고루틴 생성
    wg.Add(1)
    go func() {
        defer wg.Done()
        defer close(stringStream)

        // 문자열 슬라이스를 순회하며 채널로 데이터를 송신
        for _, s := range []string{"a", "b", "c"} {
            select {
                case <-done: // 작업을 중간에 멈추라는 종료 신호가 오면 즉시 리턴
                    return
                case stringStream <- s: // 채널이 비어있으면 문자열을 전송
            }
        }
    }()
}
```

```

// 하류 소비자 레벨에서 채널 스트림 데이터를 받아와서 출력
for result := range stringStream {
    fmt.Printf("Received from stream: %v\n", result)
}
wg.Wait()
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

Received from stream: a
Received from stream: b
Received from stream: c

```

(2) 시나리오2: 종료될 때까지 무한 반복하기

고루틴 일꾼을 백그라운드에 띄워두고, 외부에서 취소 신호(done)를 주기전까지 자기 할 일을 무한히 뱅뱅이 돌아가며 수행하게 만드는 아주 대중적인 방식이다.

이 시나리오는 코드를 배치하는 스타일(디자인 취향)에 따라 두 가지 변형으로 나뉜다. 두 방식 모두 실무에서 똑같이 정상 작동하므로 편한 방식을 선택하면 된다.

```

package main

import (
    "fmt"
    "time"
)

func main() {
    done := make(chan interface{})

    go func() {
        // 백그라운드 고루틴이 무한 루프를 돌며 일하도록 설정
        workCounter := 0
        for {
            select {
            case <-done: // 채널이 닫히면 즉시 일꾼 함수를 종료
                return
            default: // 채널이 안 닫혔으면 블로킹되지 않고 select를 그냥 통과함
            }

            // [여기서 멈출 수 없는 실제 작업을 수행합니다]
            workCounter++
            fmt.Printf("A-Style Worker doing task #%d\n", workCounter)
            time.Sleep(1 * time.Second)
        }
    }()
}

```

```

// 메인 함수에서 3초 동안 일하게 놔두다가 종료 신호를 보냄
time.Sleep(3 * time.Second)
close(done)
time.Sleep(500 * time.Millisecond) // 출력 로그가 꼬이지 않도록 잠시 대기
fmt.Println("A-Style Main process exit.")
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

A-Style Worker doing task #1
A-Style Worker doing task #2
A-Style Worker doing task #3
A-Style Main process exit.

```

donec 채널이 닫히지 않은 상태라면 select문은 대기없이 default 구문을 타고 툇 떨어져서 아래쪽에 나열된 로컬 작업 코드를 정직하게 수행 한 뒤, for문에 의해 다시 위로 올가기를 반복한다.

변형B: 임계 영역을 select문의 default 블록에 매립하기

두 번째 스타일은 해야 할 작업 코드 자체를 아예 select 문의 default 절 내부 공간에 완전히 매립시켜 가두는 방식이다. 전체 코드를 확인해보자.

```

package main
import (
    "fmt"
    "time"
)

func main() {
    done := make(chan interface{})

    go func() {
        workCounter := 0
        for {
            select {
            case <-done: // 종료 신호 감지 시 즉시 리턴
                return
            default:
                // [여기서 멈출 수 없는 실제 작업을 수행합니다]
                workCounter++
                fmt.Printf("B-Style Worker doing task #%dWn", workCounter)
                time.Sleep(1 * time.Second)
            }
        }
    }()

    // 메인 함수에서 3초 동안 일하게 놔두다가 종료 신호를 보냄
    time.Sleep(3 * time.Second)
    close(done)
    time.Sleep(500 * time.Millisecond) // 출력 로그가 꼬이지 않도록 잠시 대기
    fmt.Println("B-Style Main process exit.")
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
B-Style Worker doing task #1  
B-Style Worker doing task #2  
B-Style Worker doing task #3  
B-Style Main process exit.
```

select 문에 진입하는 순간 done 채널에 아무런 취소 신호가 들어와 있지 않다면, 런타임은 대기없이 즉시 default 내부 블록을 실행 구역으로 선택하여 처리한다.

(3) 결론

이 for-select 루프 패턴은 동시성 고루틴의 생명 주기를 부모 프로세스가 완벽하게 통제하면서도, 자원을 효율적으로 소모하며 무한 연산을 수행할 수 있도록 돕는 Go언어의 가장 뼈대가 되는 기본 기법이다. 소스코드 어디를 뒤져보아도 밥 먹듯이 발견되는 흔한 구조이므로, 눈감고도 타이핑할 수 있을 정도로 완벽하게 구조를 익혀두는 것이 좋다.

[4-3] 고루틴 누수 방지

(1) 고루틴 누수란 무엇인가?

Go 언어에서는 고루틴은 매우 값싸고 간편하게 만들 수 있는 큰 장점이 있다. Go 런타임이 알아서 수많은 고루틴을 운영체제 스레드 위로 최적화하여 분배(Multiplexing)해 주기 때문에 개발자는 하드웨어 수준의 제어를 크게 고민하지 않아도 된다.

하지만 고루틴도 엄연히 시스템 메모리 자원을 소모한다. 더욱 중요한 사실은, 고루틴은 Go의 가비지 컬렉터(GC)가 자동으로 수집하여 청소해 주지 않는다는 점이다. 고루틴 하나가 용량이 아무리 몇 킬로바이트 수준으로 작다고 하더라도, 끝나지 않고 메모리에 덩그러니 남아서 방치된다면 결국 시스템 전체의 자원을 야금야금 갉아먹게 된다. 이를 고루틴 누수(Goroutine Leak)라고 부른다.

고루틴이 정상적으로 종료되는 경로는 다음과 같이 크게 세 가지가 있다.

- 자기가 해야 할 일을 모두 마쳤을 때
- 회복 불가능한 에러를 만나 더 이상 일을 진행할 수 없을 때
- 부모 프로세스로부터 “이제 일을 그만하라”는 중단 신호를 받았을 때

앞의 두 가지 경로는 알고리즘 로직에 의해 자연스럽게 처리되지만,

세 번째인 ‘작업 취소’를 설계하는 것은 고성능 아키텍처를 위해 가장 중요하다. 고루틴들은 대개 서로 얹혀서 유기적으로 일하기 때문에, 부모 고루틴(주로 메인 함수)이 전체적인 상황을 보고 자식 고루틴들에게 안전하게 퇴근하라고 명령을 내릴 수 있어야 한다.

먼저 고루틴 누수가 발생하는 전형적인 나쁜 예시를 생략 없는 전체 코드로 확인해보자.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    doWork := func(strings <-chan string) <-chan interface{} {
        completed := make(chan interface{})
        go func() {
            defer fmt.Println("doWork exited.")
            defer close(completed)

            for s := range strings {
                // 데이터를 받아서 처리하는 공간
                fmt.Println(s)
            }
        }()
        return completed
    }

    // 대참사 원인: strings 매개변수에 무작위 데이터를 줄 생각 없이 nil 채널을 주입함
    doWork(nil)
```

```

fmt.Println("Main goroutine is doing other work...")
time.Sleep(1 * time.Second)
fmt.Println("Done.")
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

Main goroutine is doing other work...
Done.

```

출력 결과를 가만히 보시면 고루틴 내부에 정의해 둔 `defer fmt.Println("doWork exited.")` 문장이 끝내 실행되지 않은 것을 볼 수 있다.

메인 함수가 `nil` 채널을 던져주었기 때문에, 자식 고루틴의 `for s := range strings` 루프 문장은 평생 오지 않을 데이터를 기다리며 그 자리에서 영원히 얼어붙어 버린 것이다.

이 예제는 프로세스가 금방 끝나서 큰 문제가 안되지만, 만약 24시간 내내 켜져 있는 대규모 실무 서버 환경이었다면 이런 유령 고루틴들이 메모리에 수천 수만 개씩 계속 쌓여가며 결국 서버를 다운시키는 심각한 `creep`(메모리 비대화) 현상을 유발하게 된다.

(2) 해결책 1: 데이터 수신 측 고루틴 누수 방지

이 문제를 우아하게 해결하는 정석적인 방법은 부모 고루틴과 자식 고루틴 사이에 '작업 중단 신호'를 전송할 수 있는 통로를 개설하는 것이다.

Go 언어의 관례상 이 신호 통로는 오직 읽기만 가능한 `done`이라는 이름의 채널로만든다. 부모가 자식을 퇴근시키고 싶을 때 이 `done` 채널을 단 한 번 닫아버리면(`close`) 대기하던 자식 고루틴들이 한 방에 깨어나 리턴하게 된다.

`done` 채널 방어벽을 구축하여 누수를 완벽하게 수리한 전체 코드를 확인해보자.

```

package main

import (
    "fmt"
    "time"
)

func main() {
    // 관례상 done 취소 채널은 언제나 함수의 첫 번째 매개변수로 배치합니다.
    doWork := func(done <-chan interface{}, strings <-chan string) <-chan interface{} {
        terminated := make(chan interface{})
        go func() {
            defer fmt.Println("doWork exited.")
            defer close(terminated)
            for {
                select {
                case s := <-strings: // 일반 데이터 수신 처리
                    fmt.Println(s)

```



```

                                case <-done: // 핵심: 부모가 done 채널을 닫으면 이 케이스가
즉시 발동하며 고루틴을 안전하게 종료시킵니다.
                                return
                                }
                                }
                                }0
                                return terminated
                                }

done := make(chan interface{})
terminated := doWork(done, nil)

go func() {
    // 백그라운드에서 1초 동안 상황을 지켜보다가 취소 신호를 발송하는 고루틴
    time.Sleep(1 * time.Second)
    fmt.Println("Canceling doWork goroutine...")
    close(done) // 취소 신호 총성 발사
}0

<-terminated // 조인 포인트: 자식 고루틴이 안전하게 청소되어 퇴근할 때까지 대기
fmt.Println("Done.")
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

Canceling doWork goroutine...
doWork exited.

```

출력 결과를 보시면 아까와 달리 “doWork exited.”가 아주 정직하게 찍힌 것을 볼 수 있다. 비록 똑같이 nil 채널을 주입받아 데이터 수신부는 꽉 막혀있지만, select문이 매 루프마다 done 취소 신호가 왔는지 철저하게 감시하고 있었기 때문에 안전하게 탈출에 성공한 것이다. 고루틴 누수를 완벽하게 박멸했다.

(3) 해결책2: 데이터 송신 측 고루틴 누수 방지

방금 다룬 수신 측 대기 현상 외에, 반대로 자식 고루틴이 채널에 데이터를 ‘집어넣으려고(Write)’ 하다가 막혀서 굼어 죽는 누수 현상도 자주 발생한다. 대참사가 일어나는 무한 랜덤 숫자 생성기 전체 코드를 먼저 확인하겠다.

```

package main

import (
    "fmt"
    "math/rand"
    "time"
)

func main() {
    newRandStream := func() <-chan int {
        randStream := make(chan int)
        go func() {

```

```

        defer fmt.Println("newRandStream closure exited.")
        defer close(randStream)
        for {
            // 무한 루프를 돌며 계속 랜덤 숫자를 채널에 밀어 넣음
            randStream <- rand.Int()
        }
    }()
    return randStream
}

randStream := newRandStream()
fmt.Println("3 random ints:")
for i := 1; i <= 3; i++ {
    fmt.Printf("%d: %dWn", i, <-randStream)
}

// 메인 함수는 3개의 숫자만 꼭 빼먹고 자기 볼일을 끝냅니다.
fmt.Println("Main function finished reading. Waiting a bit...")
time.Sleep(1 * time.Second)
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

3 random ints:
1: 5577006791947779410
2: 8674665223082153551
3: 6129484611666145821
Main function finished reading. Waiting a bit...

```

이번에도 역시 자식 고루틴의 종료 로그가 찍히지 않는다. 메인 함수는 딱 3번만 데이터를 읽고 반복문을 끝내버렸는데, 백그라운드 자식 일꾼은 네 번째 숫자를 채널에 넣으려고 `randStream <- rand.Int()` 문장에서 평생 대기 상태로 갇혀버렸기 때문이다. 하류에서 더 이상 꺼내 읽어줄 소비자가 없으니 채널 창고가 막힌 것이다.

이 송신 측 누수 문제 역시 동일하게 done 채널을 줘주어 해결할 수 있다. 데이터를 넣는 행위 자체를 select 문의 하나의 case로 감싸고, 다른 case에 done 취소 신호를 대기시키는 설계이다. 전체코드로 확인하자.

```

package main

import (
    "fmt"
    "math/rand"
    "time"
)

func main() {
    newRandStream := func(done <-chan interface{}) <-chan int {
        randStream := make(chan int)
        go func() {
            defer fmt.Println("newRandStream closure exited.")

```

```

        defer close(randStream)
        for {
            select {
                case randStream <- rand.Int(): // 채널에 공간이 있어서 전송 성
공 시 루프 계속 진행
                case <-done: // 핵심: 데이터를 넣으려 대기하는 중에도 부모가
done을 닫으면 즉시 리턴하여 탈출!
                    return
            }
        }
    }()
    return randStream
}

done := make(chan interface{})
randStream := newRandStream(done)

fmt.Println("3 random ints:")
for i := 1; i <= 3; i++ {
    fmt.Printf("%d: %dWn", i, <-randStream)
}

// 3개의 데이터를 다 소비했으니, 자식 일꾼에게 퇴근하라고 채널을 닫아줍니다.
close(done)

// 자식 고루틴이 안전하게 퇴근할 시간을 잠시 벌어주며 상태 확인
time.Sleep(1 * time.Second)
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

3 random ints:
1: 5577006791947779410
2: 8674665223082153551
3: 6129484611666145821
newRandStream closure exited.

```

출력 결과 맨 아랫줄을 보면 마침내 로그가 깨끗하게 출력된 것을 확인할 수 있다. 채널 창고가 막혀서 꿈쩍 못 하던 송신 고루틴의 부모의 close(done) 신호를 받고 무사히 탈출한 것이다.

(4) 결론

고루틴 누수를 방지하는 핵심 철학이자 대원칙을 마음속에 꼭 새겨두어야 한다.

Go동시성 프로그래밍의 철칙

“만약 어떤 고루틴(부모)이 새로운 고루틴(자식)을 탄생시킬 책임을 지고 있다면, 그 부모는 해당 자식 고루틴을 언제든지 확실하게 중단시킬 수 있는 방법(취소 신호 수단)까지 함께 제공할 책임을 무조건 져야 한다.

이 원칙을 철저히 지키며 코드를 설계해야 프로그램의 덩치가 커져도 레고 블록처럼 안전하게 결합할 수 있고 대규모 웹 스케일 환경에서도 무너지지 않는 탄탄한 시스템 수평 확장을 이뤄낼 수 있다.

[4-4] or-channel

Go언어의 고급 동시성 설계 패턴 중, 여러 개의 취소(중단) 신호 채널들을 하나로 묶어 관리하는 or-채널 패턴에 대해 알아보겠다.

(1) or-channel 패턴의 개념

실무에서 대규모 동시성 시스템을 구축하다 보면, 여러 하위 모듈로부터 각각 독립된 done 취소 채널을 건네받을 때가 많다. 이때 우리가 원하는 논리는 다음과 같다.

“전달받은 여러 개의 취소 채널 중, 단 하나라도 먼저 닫히는(close) 놈이 발생하면 전체 시스템을 즉시 안전하게 중단시켜라.”

이 조율 작업을 하기 위해 매번 수십 줄짜리 복잡한 select 문을 작성하는 것은 매우 번거롭다. 더욱 치명적인 문제는, 런타임(실행 중) 상황에 따라 취소 채널이 3개가 들어올지, 5개가 들어올지 그 개수를 미리 알 수 없는 가변적인 환경일 때이다. 소스코드 상에 case 문 개수를 고정해 둘 수 가 없기 때문이다.

이 문제를 우아한 한 줄짜리 함수 호출 형태로 해결해 주는 아키텍처가 바로 or-channel 패턴이다. 이 패턴은 ‘재귀 함수’와 ‘고루틴’의 특성을 치밀하게 결합하여, 인자로 넘어오는 채널의 개수와 상관없이 동적으로 트리(Tree) 구조의 동시성 방어벽을 완성해낸다.

(1) or-channel 재귀 함수 소스코드 구현

가변 채널 슬라이스를 입력받아 단 하나의 복합 취소 채널을 반환하는 or 함수의 생략 없는 전체 코드를 확인해보겠다.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    // 가변 인자(variadic slice)로 취소 채널들을 전달받아 단 하나의 채널로 합쳐주는 변수 선언
    var or func(channels ...<-chan interface{}) <-chan interface{}

    or = func(channels ...<-chan interface{}) <-chan interface{} {
        // 기저 조건 1 (Termination Criteria): 들어온 채널이 아예 없으면 nil 채널 반환
        switch len(channels) {
        case 0:
            return nil
        // 기저 조건 2 (Termination Criteria): 들어온 채널이 딱 1개면 재귀를 돌 필요 없이 그 채널을 그대로 반환
        case 1:
            return channels[0]
        }
    }
```

```

orDone := make(chan interface{})

// 메인 로직을 고루틴으로 실행하여 호출 측 흐름이 대기(Blocking)하지 않도록
처리함
go func() {
    defer close(orDone)

    switch len(channels) {
        // 최적화 구간: 채널이 정확히 2개만 남았을 때는 단순 select 구문으로
        고루틴 소모를 최소화함
        case 2:
            select {
            case <-channels[0]:
            case <-channels[1]:
            }
            // 재귀 트리 구축 구간: 채널이 3개 이상일 때 작동
            default:
                select {
                case <-channels[0]:
                case <-channels[1]:
                case <-channels[2]:
                    // 3번째 인덱스 이후의 모든 채널들을 다시 or 함수에 집어넣어
                    거대한 재귀 트리를 조립함
                    // 이때 현재 계층의 orDone 채널도 함께 넘겨주어, 상위 트리가
                    종료되면 하위 트리도 연쇄 퇴근하게 만들
                    case <-or(append(channels[3:], orDone)...):
                    }
                }
            }0

        return orDone
    }

// 실행 검증을 위한 예제 작성
sig := func(after time.Duration) <-chan interface{} {
    c := make(chan interface{})
    go func() {
        defer close(c)
        time.Sleep(after)
    }0
    return c
}

start := time.Now()
// 여러 타임아웃 채널 중 가장 짧은 1초짜리 채널이 신호를 터트리는 상황 시뮬레이션
<-or(
    sig(2 * time.Hour),
    sig(5 * time.Minute),
    sig(1 * time.Second), // 이 채널이 1초 뒤 가장 먼저 닫힙니다.
    sig(1 * time.Hour),
    sig(1 * time.Minute),
)

```

```
}
    fmt.Printf("done after %vWn", time.Since(start))
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
done after 1.000216772s
```

(3) 작동 원리와 파급 효과 분석

출력 결과를 보면 2시간, 5분, 1시간짜리 무지막지하게 긴 대기 채널들을 함께 집어넣었음에도 불구하고, 가장 짧은 1초짜리 닫히는 순간 or 함수가 반환한 복합 채널 전체가 즉시 정상적으로 차단 해제(close) 된 것을 확인할 수 있다.

or 함수가 재귀적으로 호출되면서 내부적으로 이진 트리 모양의 고루틴 연결망을 형성하기 때문에 이런 제어가 가능하다. 트리 구조 내부의 임의의 위치에 있는 채널 하나가 종료 신호를 터트리면, 그 신호가 상위 부모 계층으로 도미노처럼 파급되면서 트리 전체 시스템의 자물쇠를 한 방에 연쇄적으로 풀어내게 된다.

이 간결한 한 줄짜리 추상화를 얻는 대가로 내부적으로 추가적인 고루틴들이 생성된다는 특징이 있다. 정확한 생성 공식은 다음과 같다.

```

$$f(x) = x / 2$$

(여기서 x는 입력된 취소 채널의 총 개수이다.)
```

채널5개를 넣으면 약2개 안팎의 가벼운 고루틴 일꾼들이 내부 조율을 위해 잠시 예약되는 셈이다. 3장에서 배운것처럼 Go언어의 고루틴은 문맥 전환과 생성 비용이 상상 이상으로 무척 값싸고 가볍기 때문에, 이 정도의 고루틴 추가 발생을 두고 시스템 성능을 걱정하는 것은 명쾌한 ‘조기 최적화의 오류’이다.

오히려 컴파일 시점에 몇 개의 채널이 들어올지 모르는 동적 환경에서는 이 재귀 패턴 외에 완벽하게 동시성 채널들을 병합할 수 있는 대안이 존재하지 않는다.

(4) 결론

이 or-channel 패턴은 복잡한 마이크로서비스 아키텍처나 시스템의 모듈과 모듈이 교차하는 위험 길목에서 아주 훌륭한 방어벽 역할을 수행한다. 호출 스택의 깊은 골짜기마다 흩어져 있는 다양한 취소 조건들을 상위 레벨에서 단 한줄로 묶어서 하류로 안전하게 흘려보낼 수 있다.

이 기법은 뒤이어 배울 Go표준 라이브러리의 핵심 아키텍처인 context 패키지의 내부 바인딩 원리와 일맥상통 하며, 대형 트래픽 환경에서 똑같은 요청을 여러 서버에 동시에 찢러본 뒤 가장 먼저 응답이 오는 놈만 취하고 나머지를 즉시 취소시키는 ‘복제된 요청’ 패턴을 구현할 때 모든 동시성 제어의 핵심 초석으로 다시 한번 중요하게 등장하게 된다.

[4-5] 에러 처리(Error Handling) 패턴

(1) 동시성 환경에서 에러 처리의 난제

동시성 프로그램을 작성할 때 많은 개발자가 여러 프로세스 간의 데이터 공유와 실행 흐름을 조율하는 데 너무 많은 시간을 쏟은 나머지, 에러가 발생한 예외 상황을 어떻게 우아하게 처리할 것인가는 쉽게 간과하곤 한다.

Go언어가 다른 언어들의 대중적인 예외 처리 모델(Try-Catch Exception 구조)을 과감히 배제했을 때 던진 메시지는 명확하다.

에러 처리 역시 프로그래밍의 매우 중요한 축이며, 우리가 핵심 알고리즘을 짤 때 쏟는 노력만큼 에러가 흐르는 경로에도 동일한 주의를 기울여야 한다는 것이다.

동시성 프로세스 환경에서는 에러를 만났을 때 다음 질문이 훨씬 더 복잡해진다.

“이 발생한 에러를 처리할 진짜 책임자는 누구인가?”

고루틴은 부모 프로세스나 다른 형제 고루틴들과 완전히 독립적으로 작동하기 때문에, 자식 고루틴 스스로가 에러를 만났을 때 “이걸 시스템 전체 시스템 관점에서 계속 진행해야하나, 아니면 여기서 멈춰야 하나?”를 올바르게 판단하기가 불가능에 가깝다.

먼저 자식 고루틴 안에서 에러를 스스로 처리하려고 하다가 엉거주춤한 상태에 갇혀버리는 전형적인 나쁜 예시 코드를 생략 없이 확인해보자.

```
package main

import (
    "fmt"
    "net/http"
)

func main() {
    checkStatus := func(
        done <-chan interface{},
        urls ...string,
    ) <-chan *http.Response {
        responses := make(chan *http.Response)
        go func() {
            defer close(responses)
            for _, url := range urls {
                resp, err := http.Get(url)
                if err != nil {
                    // 대참사 원인: 자식 고루틴이 에러를 외부로 보낼 방법이
                    // 없어서 혼자 화면에 출력하고 맙니다.
                    fmt.Println(err)
                    continue
                }
                select {
                case <-done:
                    return
                case responses <- resp:
                }
            }
        }()
        return responses
    }
}
```



```

        }
    }()
    return responses
}

done := make(chan interface{})
defer close(done)

urls := []string{"https://www.google.com", "https://badhost"}
for response := range checkStatus(done, urls...) {
    fmt.Printf("Response: %v\n", response.Status)
}
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

Response: 200 OK
Get "https://badhost": dial tcp: lookup badhost on 127.0.0.1:53: no such host

```

출력 결과를 보면 자식 고루틴은 에러가 났음을 알리기 위해 자신이 할 수 있는 최선의 행동인 `fmt.Println(err)`을 수행했다. 하지만 자식 고루틴은 에러 데이터를 부모에게 되돌려줄 방도가 없기 때문에 “누군가 이 로그 화면을 봐주겠지”라며 무책임하게 다음 루프로 넘어가 버린다.

이 고루틴은 “에러가 몇 번 이상 나면 전체 요청을 중단해야 하는지”, “계속 진행하는게 맞는지”에 대한 대승적인 판단을 전혀 내릴 수 없다. 고루틴을 이런 어색하고 수동적인 포지션에 방치해서는 안된다.

(2) 해결책: 결과와 에러를 하나로 묶는 관심사 분리

이 문제를 해결하는 올바른 정석 설계는 ‘관심사의 분리’이다. 자식 고루틴은 오직 연산을 수행하고 그 연산에서 발생한 결과물 혹은 에러를 있는 그대로 상류로 쏘아 올려주는 역할만 전담하게 만든다.

그리고 시스템 전체 컨텍스트와 운영 상태를 완벽하게 파악하고 있는 부모 고루틴(이 경우 메인 함수)이 그 에러를 받아 판별하고 똑똑한 의사 결정을 내리게 구조를 바꾸는 것이다.

이를 구현하기 위해 정상 응답 결과(`*http.Response`)와 에러 정보(`error`)를 단단하게 한 몸으로 결합한 `Result` 구조체 타입을 새로 정의하여 채널로 유통해야 한다.

전체 코드를 확인해보자.

```
package main

import (
    "fmt"
    "net/http"
)

// 핵심: 연산 결과물과 에러 가능성을 하나의 스펙으로 묶어주는 전용 결과 구조체 정의
type Result struct {
    Error    error
    Response *http.Response
}

func main() {
    checkStatus := func(done <-chan interface{}, urls ...string) <-chan Result {
        results := make(chan Result)
        go func() {
            defer close(results)
            for _, url := range urls {
                resp, err := http.Get(url)
                // 발생한 에러든 성공한 응답이든 가리지 않고 결과 인스턴스에
                통째로 담아 보냅니다.
                result := Result{Error: err, Response: resp}
                select {
                case <-done:
                    return
                case results <- result:
                }
            }
        }()
        return results
    }

    done := make(chan interface{})
    defer close(done)

    urls := []string{"https://www.google.com", "https://badhost"}

    // 메인 고루틴(부모)은 수신한 Result 객체를 열어보고 대응적인 컨텍스트 하에 판단을
    내립니다.
    for result := range checkStatus(done, urls...) {
        if result.Error != nil {
            fmt.Printf("error: %v\n", result.Error)
            continue
        }
        fmt.Printf("Response: %v\n", result.Response.Status)
    }
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
Response: 200 OK
error: Get "https://badhost": dial tcp: lookup badhost on 127.0.0.1:53: no such host
```

(3) 유통 에러 패턴의 실무 확장성

이렇게 에러 처리를 자식 고루틴 내부에서 전부르게 털어버리지 않고 부모 고루틴으로 온전하게 유통해 오면, 동시성 프로그램에서도 우리가 평소에 작성하던 친숙하고 명확한 Go언어의 에러 핸들링 패턴을 그대로 구사할 수 있게 된다.

예를 들어, 프로그램의 요구사항이 바뀌어 “동시성 네트워크 요청을 처리하던 중 에러가 통산 3번 이상 누적되면, 위험 상황으로 판단하고 하위 고루틴 일꾼들을 전부 강제 퇴근(취소)시킨 뒤 전체 루프를 즉시 깨부수고 탈출하라”라는 복잡한 동적 비즈니스 규칙이 추가되어도 아주 손쉽게 코드를 작성할 수 있다.

확장된 제어 전체 코드를 확인해보자.

```
package main

import (
    "fmt"
    "net/http"
)

type Result struct {
    Error    error
    Response *http.Response
}

func main() {
    checkStatus := func(done <-chan interface{}, urls ...string) <-chan Result {
        results := make(chan Result)
        go func() {
            defer close(results)
            for _, url := range urls {
                resp, err := http.Get(url)
                result := Result{Error: err, Response: resp}
                select {
                case <-done:
                    return
                case results <- result:
                }
            }
        }()
        return results
    }

    done := make(chan interface{})
    // 메인 함수가 중간에 break loop로 탈출하더라도 defer close(done) 신호에 의해
    // 백그라운드에 남아있던 자식 고루틴들이 고루틴 누수 없이 즉시 안전하게 종료됩니다.
```

```

defer close(done)

errCount := 0
// 의도적으로 프로토콜 에러를 유발할 "a", "b", "c", "d" 문자열 주입
urls := []string{"a", "https://www.google.com", "b", "c", "d"}

for result := range checkStatus(done, urls...) {
    if result.Error != nil {
        fmt.Printf("error: %v\n", result.Error)
        errCount++

        // 부모 고루틴이 에러 누적 카운트를 세어 동적으로 전체 시스템 흐름을
        // 통제함
        if errCount >= 3 {
            fmt.Println("Too many errors, breaking!")
            break
        }
        continue
    }
    fmt.Printf("Response: %v\n", result.Response.Status)
}
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

error: Get "a": unsupported protocol scheme ""
Response: 200 OK
error: Get "b": unsupported protocol scheme ""
error: Get "c": unsupported protocol scheme ""
Too many errors, breaking!

```

(4) 결론

고루틴 동시성 아키텍처를 빌드업할 때 가슴 깊이 명심해야 할 핵심 교훈은 다음과 같다.

Go 동시성 에러 처리의 대원칙

“동시성 작업 환경에서 에러는 일반 데이터값과 완벽하게 동등한 ‘일급 시민’으로 대우받아야 한다”

내가 설계한 고루틴이 내부 연산 과정에서 에러를 뿔어낼 가능성이 단 1%라도 존재한다면, 그 에러는 무조건 최종 성공 결과물 타입과 실과 바늘처럼 뽕뽕 묶어서 동일한 채널 소통 라인을 타고 상류로 배달되어야 한다.

우리가 평소에 작성하던 일반적인 동기식 함수들이 (결과값, error) 형태로 두 개의 값을 정직하게 반환하며 책임을 위로 올리던 방식과 완벽히 동일하게 말이다.
이 원칙을 지켜야만 프로그램이 커져도 소통 구역이 꼬이지 않고 깨끗한 예외 복구 아키텍처를 유지할 수 있다.

[4-6] 파이프라인

(1) 파이프라인 패턴의 개념

우리가 프로그램을 작성할 때, 수천 줄짜리 거대한 함수 하나에 모든 로직을 몰아서 짜지는 않는다. 우리는 함수, 구조체, 메서드 등의 형태로 끊임없이 ‘추상화’를 진행한다.

그 이유는 시스템의 거대한 실행 흐름에 지장을 주지 않으면서 특정 코드 영역의 세부 구현을 감추고, 다른 영역에 부작용을 주지 않은 채 독립적으로 코드를 수정하기 위함이다. 만약 하나의 논리적 요구사항을 고치기 위해 시스템 여기저기를 다 건드려야 한다면, 그 시스템은 추상화 설계가 무너진 상태라고 볼 수 있다.

파이프라인은 시스템의 추상화를 구축하는 매우 강력한 도구이다. 특히 대량의 데이터 배치(Batches)나 지속해서 흘러들어오는 데이터 스트림(Streams)를 연속적으로 가공하고 처리해야 할 때 압도적인 위력을 발휘한다.

원래 파이프라인이라는 단어는 1856년 석유나 액체를 한 곳에서 다른 곳으로 실어 나르는 수송관에서 유래한 말이다. 컴퓨터 과학에서는 액체 대신 ‘데이터’를 이동시킨다는 개념으로 이 용어를 빌려왔다.

파이프라인 아키텍처는 “데이터를 입력받아 특정 연산을 수행한 뒤, 가공된 데이터를 다음 단계로 다시 토스하는 독립된 기능들의 연속적인 연결 고리”에 불과하다. 이때 각각의 독립적인 연산 단계를 파이프라인의 스테이지(Stage)라고 부른다.

파이프라인을 사용하면 각 스테이지의 관심사가 완벽히 분리되므로 다음과 같은 수많은 이점을 얻을 수 있다.

- 다른 스테이지에 영향을 주지 않고 특정 스테이지의 내부 로직만 독립적으로 수정할 수 있다.
- 스테이지 자체를 전혀 고치지 않고, 스테이지들을 조립하고 엮는 결합 순서를 자유롭게 믹스앤매치할 수 있다.
- 상류와 하류의 스테이지들이 서로 멈추지 않고 완전히 동시에 데이터를 처리할 수 있게 만든다.
- 부하가 많이 걸리는 특정 스테이지 구역만 일꾼 고루틴을 대거 투입해 다중 분산 연산을 처리하는 팬아웃을 구현하거나, 초당 처리량을 통제하는 속도 제한(Rate-limiting) 기술을 손쉽게 녹여낼 수 있다.

(2) 절차적 함수로 구현하는 배치 처리 스테이지

파이프라인 스테이지의 대원칙은 아주 단순하다. 데이터를 받아서, 변형하고, 다시 뱉어내는 것이다.

먼저 슬라이트 데이터를 한꺼번에 받아서 가공하는 고전적인 배치 처리 방식의 파이프라인 스테이지 함수들을 작성한다. 숫자를 곱해주는 스테이지(multiply)와 숫자를 더해주는 스테이지(add)를 정직하게 구현한 전체 코드이다.

```
package main

import (
    "fmt"
)

func main() {
    // 1. 입력받은 모든 슬라이스 값에 인자를 곱해주는 파이프라인 스테이지 정의
    multiply := func(values []int, multiplier int) []int {
        multipliedValues := make([]int, len(values))
        for i, v := range values {
            multipliedValues[i] = v * multiplier
        }
        return multipliedValues
    }

    // 2. 입력받은 모든 슬라이스 값에 인자를 더해주는 파이프라인 스테이지 정의
    add := func(values []int, additive int) []int {
        addedValues := make([]int, len(values))
        for i, v := range values {
            addedValues[i] = v + additive
        }
        return addedValues
    }

    ints := []int{1, 2, 3, 4}

    // 3. 두 개의 스테이지를 마치 하나의 파이프라인 수송관처럼 연결하여 실행함
    // (ints 데이터 -> 곱하기 2 스테이지 -> 더하기 1 스테이지 -> 최종 출력)
    for _, v := range add(multiply(ints, 2), 1) {
        fmt.Println(v)
    }
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
3
5
7
9
```

평범한 동기식 함수처럼 보이지만, 이 함수들은 파이프라인 스테이지가 가져야 할 완벽한 두 가지 속성을 충족하고 있다.

파이프라인 스테이지의 2대 성립 조건

1. 하나의 스테이지는 ‘소비하는 데이터 타입’과 ‘반환하는 데이터 타입’이 완벽하게 일치해야 한다. (이 경우 []int를 받아 []int를 반환함)
2. 스테이지 자체가 언어에 의해 ‘일급 객체’로 취급되어, 변수에 할당되거나 여기저기 매개변수로 자유롭게 던져질 수 있어야 한다. Go 언어의 함수는 완벽하게 일급 객체이므로 이 조건에 아주 잘 들어맞는다.

이 조건 덕분에 우리는 기존 함수들을 단 한줄도 수정하지 않고, 외부 조립 라인에서 파이프라인의 길이를 무한하게 확장할 수 있다.

예를 들어, 연산 요구사항이 바뀌어 뒤에 “곱하기 2 작업을 한 번 더 추가해라.”라고 한다면, 기존 코드는 그대로 둔 채 겹 표면을 multiply 스테이지로 한 번 더 감싸주기만 하면된다.

전체 코드로 보자.

```
package main

import (
    "fmt"
)

func main() {
    multiply := func(values []int, multiplier int) []int {
        multipliedValues := make([]int, len(values))
        for i, v := range values {
            multipliedValues[i] = v * multiplier
        }
        return multipliedValues
    }

    add := func(values []int, additive int) []int {
        addedValues := make([]int, len(values))
        for i, v := range values {
            addedValues[i] = v + additive
        }
        return addedValues
    }

    ints := []int{1, 2, 3, 4}

    // 스테이지의 구조적 변경 없이 외부 조립만으로 파이프라인 연산 확장
    // (ints -> 곱하기 2 -> 더하기 1 -> 곱하기 2 -> 최종 출력)
    for _, v := range multiply(add(multiply(ints, 2), 1), 2) {
        fmt.Println(v)
    }
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
6
10
14
18
```

(3) 메모리 절약을 위한 스트림 처리 스테이지

방금 작성한 방식은 전체 덩어리를 한꺼번에 처리하는 배치 처리 파이프라인이었다. 데이터 원본을 오염시키지 않고 깨끗하게 연산하기 위해, 매 스테이지마다 `make([]int, len(values))` 문장을 수행하며 배열 메모리를 매번 새로 복사해 내고 있다.

이 방식은 치명적인 단점이 있다.

파이프라인 스테이지가 한 단계 늘어날 때마다 메모리 사용량이 입력 데이터 크기만큼 배수로 계속 켜중켜중 증가한다는 점이다.

기가바이트 단위의 대용량 데이터를 다룬다면 서버 메모리가 순식간에 터져버릴 것이다.

이 메모리 폭증 문제를 해결하는 다른 선택지가 바로 데이터를 한 개씩 쪼개어 연속적으로 흘려보내는 스트림 처리 방식이다.

데이터 슬라이스 통째로 넘기지 않고 정수 값 딱 한개(int)만 받아서 즉시 처리하고 반환하는 가벼운 구조이다. 전체코드로 확인해보자.

```
package main

import (
    "fmt"
)

func main() {
    // 단 하나의 단편적인 데이터값만 받아 즉시 처리하는 스트림 형태의 스테이지 정의
    multiply := func(value, multiplier int) int {
        return value * multiplier
    }

    add := func(value, additive int) int {
        return value + additive
    }

    ints := []int{1, 2, 3, 4}

    // 데이터 원본 순회 힙 매커니즘을 외부 for문에게 전적으로 일임함
    for _, v := range ints {
        fmt.Println(multiply(add(multiply(v, 2), 1), 2))
    }
}
```


위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
6
10
14
18
```

(4) 절차적 스트림 처리의 한계와 동시성 아키텍처의 필요성

스트림 처리 방식으로 코드를 변경했더니, 매 단계마다 거대한 슬라이스 배열 메모리를 새로 복사해 내던 오버헤드가 완벽하게 사라졌다. 프로그램이 사용하는 메모리 총량이 파이프라인에 진입하는 단 한 개의 데이터 크기 수준으로 뚝 떨어졌다.

하지만 이 단순한 절차적 스트림 구조는 코드를 한 단계 확대해 보면 심각한 설계적 결함과 한계점들을 드러내기 시작한다.

- 낮은 재사용성과 확장성 제한

파이프라인에 데이터를 하나씩 급여하는 무거운 책임 처리를 외부 for-range 문이 직접 수행하고 있다. 파이프라인의 결함 형태가 for문 몸체 내부에 완전히 굳어버려(Hard-coded), 스테이지의 조합을 동적으로 바꾸거나 조립 라인을 재사용하기가 무척 까다롭다.

- 과도한 함수 호출 비용

루프가 한 바퀴 돌 때마다 매번 3번의 함수 호출이 동기식으로 일어난다. 1,000만 번 루프를 돈다면 3,000만 번의 함수 호출 오버헤드가 프로세스를 느리게 만든다.

- 동시성 제어의 불가능

이 구조는 완벽한 단일 스레드 동기식 코드이다. 첫 번째 데이터가 곱하기, 더하기, 곱하기 연산을 모두 마치고 출력이 끝날 때까지 두 번째 데이터는 일을 시작조차 하지 못하고 대기해야 한다. 멀티코어 CPU 장비의 자원을 활용해 “1번 스테이지 일꾼이 첫 번째 데이터를 가공해서 2번 스테이지에게 토스하자마자, 곧 바로 두 번째 데이터를 가져와서 동시에 연산하는 비동기식 컨베이어 벨트 소통 구조”를 구현할 수 없다.

(5) 결론

이 평범한 동기식 함수들은 파이프라인의 사상과 개념을 이해하는 도구로는 제 역할을 다했다.

하지만 우리가 진짜 원하는 웹 스케일 급의 수평 확장성과 고성능 멀티코어 비동기 파이프라인 아키텍처를 구현하기 위해서는, 이 단일 스레드 절차적 루프 구조를 완벽하게 깨부수어야 한다. 각 스테이지 단계를 독립된 고루틴 일꾼들로 독립시키고, 그 사이를 흐르는 강줄기를 Go언어의 최고 무기인 채널(chan)기본 도구로 전면 전환해야 한다.

[4-7] 파이프라인을 구축하는 정석적인 모범 설계 기법

Go 언어의 채널은 파이프라인 스테이지를 조립하는 데 완벽하게 들어맞는 특성을 가지고 있다. 채널은 데이터를 주고받을 수 있고, 동시성 환경에서 안전하며, for range 문으로 순회할 수 있고, 언어의 일급 객체로 취급되기 때문이다.

(1) 채널 기반 동시성 파이프라인 전체 소스코드

독립된 고루틴 일꾼들이 채널 수송관을 통해 데이터를 비동기식으로 실어 나르는 정석적인 파이프라인 구현 전체 코드이다.

```
package main

import (
    "fmt"
)

func main() {
    // 1. 기초 데이터를 채널 스트림으로 변환해 주는 제너레이터 스테이지
    generator := func(done <-chan interface{}, integers ...int) <-chan int {
        intStream := make(chan int)
        go func() {
            defer close(intStream)
            for _, i := range integers {
                select {
                    case <-done:
                        return
                    case intStream <- i:
                }
            }
        }()
        return intStream
    }

    // 2. 상류 채널에서 데이터를 받아 숫자를 곱해주는 파이프라인 스테이지
    multiply := func(
        done <-chan interface{},
        intStream <-chan int,
        multiplier int,
    ) <-chan int {
        multipliedStream := make(chan int)
        go func() {
            defer close(multipliedStream)
            for i := range intStream {
                select {
                    case <-done:
                        return
                    case multipliedStream <- i * multiplier:
                }
            }
        }()
        return multipliedStream
    }
}
```

```

// 3. 상류 채널에서 데이터를 받아 숫자를 더해주는 파이프라인 스테이지
add := func(
    done <-chan interface{},
    intStream <-chan int,
    additive int,
) <-chan int {
    addedStream := make(chan int)
    go func() {
        defer close(addedStream)
        for i := range intStream {
            select {
            case <-done:
                return
            case addedStream <- i + additive:
            }
        }
    }()
    return addedStream
}

// 4. 전체 고루틴의 생명주기를 통제할 done 채널 선언
done := make(chan interface{})
defer close(done)

// 5. 파이프라인 수송관 조립 및 데이터 주입
intStream := generator(done, 1, 2, 3, 4)
pipeline := multiply(done, add(done, multiply(done, intStream, 2), 1), 2)

// 6. 최종 하류에서 데이터를 하나씩 추출하여 출력
for v := range pipeline {
    fmt.Println(v)
}
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

6
10
14
18

```

(2) 우리가 동시성 파이프라인을 통해 얻은 이점

이전의 단순한 절차적 코드에 비해 소스코드의 양은 확연히 늘어났다.
그렇다면 우리가 이 구조를 통해 얻은 진정한 기술적 이점은 무엇일까?
코드를 세부적으로 쪼개어 분석하겠다.

코드1: 제너레이터(Generator) 패턴의 확립

파이프라인의 출발점에는 언제나 일반 슬라이스나 가변 인자 같은 일반 데이터를
채널 스트림으로 변환해주는 작업이 필요하다.

```
intStream := generator(done, 1, 2, 3, 4)
```

위의 generator 함수는 가변 인자로 정수 배열을 받아 내부 고루틴을 통해 값을 채널에 하나씩
밀어 넣어준다. 이처럼 이산적인 데이터 셋을 채널의 데이터 스트림으로 전환해 주는 함수를
동시성 아키텍처에서는 제너레이터(Generator)라고 부르며, 모든 채널 기반 파이프라인의
필수 진입점 역할을 수행한다.

코드2: 완벽한 비동기 컨베이어 벨트식 동시성 실행

가장 혁신적인 변화는 파이프라인의 모든 스테이지 단계가 완전히 동시에 실행된다는 점이다.

```
pipelien := multiply(done, add(done, multiply(done, intStream, 2), 1), 2)
```

기존의 절차적 루프 코드에서는 첫 번째 데이터가 모든 계산(multiply → add → multiply)을
완전히 마치고 화면에 출력될 때까지 두 번째 데이터는 연산을 시작조차 하지 못하고 대기해야 한
다.

반면 채널 기반 파이프라인에서는 각 스테이지가 독립된 고루틴 일꾼들로 채워져 있다.
1번 스테이지 일꾼이 첫 번째 데이터를 계산해서 2번 스테이지 채널에 던지자마자,
메인 함수가 이를 출력하기도 전에 1번 스테이지 일꾼은 곧바로 두 번째 데이터를 가져와
비비기 시작한다. 상류와 하류의 작업자들이 서로를 기다리며 노는 시간 없이 컴퓨터의 멀티코어
두뇌를 분산 활용하여 연산을 병렬적으로 처리하게 되는 것이다.

(3) 데이터가 파이프라인을 타고 흐르는 연산 타임라인 분석

데이터가 각 채널 수송관을 통과할 때 내부적으로 어떤 연산이 일어나는지 타임라인 흐름표를
통해 정밀하게 파악하겠다.

아래의 숫자는 데이터가 파이프라인 스테이지에 진입하는 순간의 실제 물리적인 값 변화이다.

루프 차례 (Iteration)	제너레이터 (Generator) 진입	1차 곱하기 (Multiply)	더하기 (Add)	2차 곱하기 (Multiply)	최종 출력값 (Value)
0번째 데이터	1	1	2	3	6
1번째 데이터	2	2	4	5	10
2번째 데이터	3	3	6	7	14
3번째 데이터	4	4	8	9	18
종료 단계	채널 닫힘 (closed)	채널 닫힘 (closed)	채널 닫힘 (closed)	채널 닫힘 (closed)	루프 탈출 (exit)

(4) 취소 신호의 도미노 파급 효과

대규모 마이크로서비스나 동시성 시스템 환경에서 유기적으로 연결된 수많은 자식 고루틴들을 중간에 안전하게 중단시키는 것은 무척 까다롭다.

만약 데이터 연산 도중 메인 함수에서 `close(done)` 명령을 내리면 시스템 내부 채널망에는 어떤 파급 효과가 일어날까?

타임아웃이나 취소 신호가 발동했을 때의 연산 타임라인을 확인해보자.

루프 차례 (Iteration)	제너레이터 (Generator)	1차 곱하기 (Multiply)	더하기 (Add)	2차 곱하기 (Multiply)	최종 출력 결과
0번째 데이터 처리	1	1	2	3	6 정상 출력
중단 신호 발동	<code>close(done)</code>	-	-	-	-
1번째 데이터 단계	채널 즉시 차단	3 수신 후 중단	4 수신 후 중단	6 수신 후 중단	안전하게 탈출

부모 고루틴이 `done` 채널을 닫아버리는 순간, 그 취소 신호는 파이프라인 전체 수송관을 타고 도미노처럼 하류로 순식간에 전파된다. 이것이 가능한 이유는 각 스테이지 코드 내부에 심어둔 두 가지 완벽한 장치 덕분이다.

- `for range` 루프의 특성:

각 스테이지는 자신보다 상류에 있는 채널을 `for range` 문으로 순회하고 있다. 상류 고루틴이 취소 신호를 받고 채널을 닫으며 퇴근하면, 하류의 `range` 문 역시 자동으로 루프를 깨고 탈출하여 연쇄 퇴근을 하게 된다.

- `select` 문과 `done` 케이스의 결합:

설령 데이터 전송 창고가 꽉 막혀서 값을 넣지 못하고 멈춰 서 있는 상황(Blocking)일지라도, `select` 문이 매 루프마다 `case <- done` 취소 신호를 삼엄하게 감시하고 있기 때문에 즉시 안전하게 함수를 탈출할 수 있다.

(5) 결론

이 추상화 뼈대 구조 덕분에 파이프라인의 시작 단계(제너레이터)부터 최종 종착지(최종 스테이지)에 이르기까지, 모든 구간 내 고루틴들은 단 한 명의 낙오자(고루틴 누수) 없이 `close(done)` 신호 한 방에 완벽하게 청소되어 리턴할 수 있게 된다.

상류가 취소되면 하류가 연쇄적으로 닫히고, 내가 막히면 `select`가 뚫어주는 수학적 귀납적 안정성이 완벽하게 성립하는 구조이다.

이 채널 기반 파이프라인 설계 아키텍처는 Go언어 동시성 디자인 패턴의 가장 핵심이 되는 중심 기둥이므로 완벽하게 구조를 기억해두는 것이 좋다.

[4-8] 실무용 제너레이터 패턴

(1) 무한 루프 스트림 제어: repeat와 take 스테이지

동시성 파이프라인을 설계할 때 가장 매력적인 기법 중 하나는 필요한 만큼 데이터를 동적으로 찍어내는 무한 스트림 구조이다. 이를 위해 두 가지 결합성 높은 스테이지를 구현한다.

- repeat 스테이지: 인자로 넘겨받은 값들을 취소 신호가 오기 전까지 채널 스트림에 무한히 반복해서 밀어 넣어주는 제너레이터이다.
- take 스테이지: 상류 채널에서 데이터가 아무리 많이 쏟아지더라도, 오직 지정된 num개수만큼만 데이터를 꺼내 읽은 뒤 수송관을 닫고 안전하게 퇴근하는 브레이크 스테이지이다.

두 스테이지가 만나 어떻게 무한의 세계를 안전한 상한선 안으로 제어하는지 전체 코드로 확인해보자.

```
package main

import (
    "fmt"
)

func main() {
    // 1. 취소 신호가 오기 전까지 값들을 무한 반복 배출하는 제너레이터 정의
    repeat := func(done <-chan interface{}, values ...interface{}) <-chan interface{} {
        valueStream := make(chan interface{})
        go func() {
            defer close(valueStream)
            for {
                for _, v := range values {
                    select {
                        case <-done:
                            return
                        case valueStream <- v:
                    }
                }
            }
        }()
        return valueStream
    }

    // 2. 상류 채널에서 정확히 지정된 개수만 징수하고 종료하는 스테이지 정의
    take := func(
        done <-chan interface{},
        valueStream <-chan interface{},
        num int,
    ) <-chan interface{} {
        takeStream := make(chan interface{})
        go func() {
            defer close(takeStream)
            for i := 0; i < num; i++ {
                select {
```

```

        case <-done:
            return
        case takeStream <- <-valueStream:
    }

    }
}()
return takeStream
}

done := make(chan interface{})
defer close(done)

// 3. 무한히 1을 배출하는 파이프라인에서 딱 10개만 추출하여 순회 출력
for num := range take(done, repeat(done, 1), 10) {
    fmt.Printf("%v ", num)
}
fmt.Println()
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
1 1 1 1 1 1 1 1 1 1
```

이 연산 구조가 매우 가볍고 효율적인 이유는 Go 채널의 블로킹(대기) 체계 덕분이다. 상류의 repeat 고루틴이 데이터를 채널에 넣으려고 하더라도, 하류의 take 고루틴이 데이터를 꺼내 가지 않으면 그 자리에 그대로 멈춰 서서 대기한다.

이론상 무한 스트림을 다루고 있지만, 실제 메모리와 CPU 연산 관점에서는 하류에서 요청한 딱 N+1번만큼만 데이터를 안전하게 계산해 내는 강력한 주문형(On-demand) 최적화 아키텍처이다.

(2) 동적 연산 제너레이터: repeatFn 스테이지

이번에는 단순한 정적 데이터 값이 아니라, 호출할 때마다 매번 새로운 연산을 수행하는 '함수'를 통째로 넘겨받아 무한 배출하는 repeatFn 제너레이터를 알아보자. 필요할 때마다 동적으로 난수를 무한 생성하는 전체 코드를 확인해 보자.

```

package main

import (
    "fmt"
    "math/rand"
)

func main() {
    take := func(
        done <-chan interface{},
        valueStream <-chan interface{},
        num int,
    ) <-chan interface{} {
        takeStream := make(chan interface{})
    }
}

```

```

        go func() {
            defer close(takeStream)
            for i := 0; i < num; i++ {
                select {
                    case <-done:
                        return
                    case takeStream <- <-valueStream:
                }
            }
        }()
    }
    return takeStream
}

```

의

// 1. 매 루프마다 주입된 함수의 실행 결과물을 채널 스트림으로 배출하는 제너레이터 정

```

repeatFn := func(
    done <-chan interface{},
    fn func() interface{},
) <-chan interface{} {
    valueStream := make(chan interface{})
    go func() {
        defer close(valueStream)
        for {
            select {
                case <-done:
                    return
                case valueStream <- fn():
            }
        }
    }()
    return valueStream
}

```

```

done := make(chan interface{})
defer close(done)

```

// 2. 난수 발생 함수를 인자로 주입함

```

randomFunc := func() interface{} { return rand.Int() }

```

// 3. 무한의 난수 채널 수송관에서 정확히 10개의 난수만 추출하여 출력

```

for num := range take(done, repeatFn(done, randomFunc), 10) {
    fmt.Println(num)
}

```

}

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
5577006791947779410
8674665223082153551
6129484611666145821
4037200794235010051
3916589616287113937
6334824724549167320
605394647632969758
1443635317331776148
894385949183117216
2775422040480279449
```

(3) 빈 인터페이스 채널 설계와 타입 변환 스테이지

우리가 작성한 이 다용도 제너레이터 스테이지들은 왜 특정 데이터 타입이 아닌 빈 인터페이스 채널을 통해 데이터를 유통하고 있을까?

파이프라인 아키텍처의 가장 큰 무기는 ‘스테이지 코드를 재사용하여 레고 블록처럼 유연하게 조립하는 능력’에서 나온다. 데이터를 반복 생성하는 repeat나 개수를 끊어주는 take 로직은 자신이 처리하는 데이터가 문자열인지 숫자인지 그 내밀한 세부 타입 정보를 전혀 알 필요가 없다.

그저 매개변수가 몇 개 들어왔는지만 알고 채널 통로만 열어주면 장땡이다.

따라서 범용 표준 파이프라인 패턴 라이브러리를 구축할 때는 빈 인터페이스 채널로 큰 물줄기를 잡는 것이 구조적으로 훨씬 이롭다.

실제 서비스 비즈니스 단에서 명확한 고유 타입이 필요한 순간이 오며, 파이프라인 중간에 ‘타입 변환 전용 스테이지’를 한 칸 끼워 넣어서 하류로 안전하게 강줄기를 틀어주면 된다. 빈 인터페이스를 확실한 string 타입 채널로 형변환 해주는 toString 스테이지 전체 코드를 확인해보자.

```
package main

import (
    "fmt"
)

func main() {
    repeat := func(done <-chan interface{}, values ...interface{}) <-chan interface{} {
        valueStream := make(chan interface{})
        go func() {
            defer close(valueStream)
            for {
                for _, v := range values {
                    select {
                        case <-done:
                            return
                        case valueStream <- v:
                    }
                }
            }
        }()
    }
}
```

```

    }0
    return valueStream
}

take := func(done <-chan interface{}, valueStream <-chan interface{}, num int) <-
chan interface{} {
    takeStream := make(chan interface{})
    go func() {
        defer close(takeStream)
        for i := 0; i < num; i++ {
            select {
            case <-done:
                return
            case takeStream <- <-valueStream:
            }
        }
    }0
    return takeStream
}

// 1. 빈 인터페이스 스트림을 받아 실제 고유 string 타입 스트림 채널로 강제 변환해 주
// 는 스테이지 정의
toString := func(
    done <-chan interface{},
    valueStream <-chan interface{},
) <-chan string {
    stringStream := make(chan string)
    go func() {
        defer close(stringStream)
        for v := range valueStream {
            select {
            case <-done:
                return
            // 인터페이스 내부 알맹이를 string 값으로 명확하게 assertion 실
            case stringStream <- v.(string):
            }
        }
    }0
    return stringStream
}

done := make(chan interface{})
defer close(done)

var message string
// 2. (무한 텍스트 배출 -> 5개 커트 -> string 전용 형변환 파이프라인 가동)
for token := range toString(done, take(done, repeat(done, "I", "am."), 5)) {
    message += token
}
fmt.Printf("message: %s...%n", message)
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
message: Iam,Iam,I...
```

(4) 범용 인터페이스 아키텍처 vs 고유 타입 아키텍처 성능 검증

일부 간간한 개발자들은 빈 인터페이스 채널을 유통하고 이를 다시 변환하는 스테이지 고루틴을 한 단계 더 추가하는 구조에 대해 “성능 저하 오버헤드가 크지 않을까?” 우려의 목소리를 내기도 한다.

이 의구심을 완전히 불식시키기 위해, 방금 구현한 범용 인터페이스 방식(BenchmarkGeneric)과 처음부터 끝까지 정적으로 뭉개 구현한 고유 타입 방식(BenchmarkTyped)의 실제 연산 처리 속도 차이를 측정하는 정밀 벤치마크 테스트 코드를 돌려보았다.

```
package main

import (
    "fmt"
    "testing"
)

// 측정 방식 A: 빈 인터페이스 유통 및 toString 스테이지를 경유하는 구조
func BenchmarkGeneric(b *testing.B) {
    repeat := func(done <-chan interface{}, values ...interface{}) <-chan interface{} {
        valueStream := make(chan interface{})
        go func() {
            defer close(valueStream)
            for {
                for _, v := range values {
                    select {
                        case <-done:
                            return
                        case valueStream <- v:
                    }
                }
            }
        }()
        return valueStream
    }

    take := func(done <-chan interface{}, valueStream <-chan interface{}, num int) <-chan interface{} {
        takeStream := make(chan interface{})
        go func() {
            defer close(takeStream)
            for i := 0; i < num; i++ {
                select {
                    case <-done:
                        return
                    case takeStream <- <-valueStream:
                }
            }
        }()
    }
}
```

```

    }
    }()
    return takeStream
}

toString := func(done <-chan interface{}, valueStream <-chan interface{}) <-chan
string {
    stringStream := make(chan string)
    go func() {
        defer close(stringStream)
        for v := range valueStream {
            select {
            case <-done:
                return
            case stringStream <- v.(string):
            }
        }
    }()
    return stringStream
}

done := make(chan interface{})
defer close(done)
b.ResetTimer()
for range toString(done, take(done, repeat(done, "a"), b.N)) {
}

// 측정 방식 B: 처음부터 끝까지 string 고유 채널로만 직통 유통하는 구조
func BenchmarkTyped(b *testing.B) {
    repeat := func(done <-chan interface{}, values ...string) <-chan string {
        valueStream := make(chan string)
        go func() {
            defer close(valueStream)
            for {
                for _, v := range values {
                    select {
                    case <-done:
                        return
                    case valueStream <- v:
                    }
                }
            }
        }()
        return valueStream
    }

    take := func(done <-chan interface{}, valueStream <-chan string, num int) <-chan
    string {
        takeStream := make(chan string)
        go func() {
            defer close(takeStream)

```

```

        for i := num; i > 0 || i == -1; {
            if i != -1 {
                i--
            }
            select {
            case <-done:
                return
            case takeStream <- <-valueStream:
            }
        }
    }()
    return takeStream
}

done := make(chan interface{})
defer close(done)
b.ResetTimer()
for range take(done, repeat(done, "a"), b.N) {
}

func main() {
    // 실제 go test -bench=. 벤치마크 가동 시뮬레이션 결과 실행 출력
    resGeneric := testing.Benchmark(BenchmarkGeneric)
    resTyped := testing.Benchmark(BenchmarkTyped)

    fmt.Printf("BenchmarkGeneric-4Wt%dWt%d ns/opWn", resGeneric.N,
resGeneric.NsPerOp())
    fmt.Printf("BenchmarkTyped-4Wt%dWt%d ns/opWn", resTyped.N,
resTyped.NsPerOp())
}

```

위 벤치마크 시뮬레이션 코드를 가동하면 다음과 같은 결과가 출력된다.

BenchmarkGeneric-4	1000000	2266 ns/op
BenchmarkTyped-4	1000000	1181 ns/op

지표 결과를 확인해 보겠다.

처음부터 굳건하게 전용 데이터 타입 채널로 밀어붙이는 방식(BenchmarkTyped)이 범용 인터페이스 구조보다 약 2배 가까이 빠른 성능 수치를 보여주고 있다. 자릿수의 절대적인 크기(나노초 단위)를 가만히 돋보기를 대고 쳐다보면 사실 그 격차는 시스템에 아무런 해를 주지 못할 정도로 아주 경미한 수준에 불과하다.

실무 대형 서비스 환경에서 동시성 파이프라인의 전체 실행 속도를 옥죄는 진짜 범인은 이런 미세한 인터페이스 변환 연산이 아니다.

대부분은 디스크에서 대용량 파일을 읽어오거나, 원격 API를 호출하는 무거운 입출력(I/O Bound) 네트워크 병목 현상이거나, 혹은 연산 자체에 무지막지한 리소스를 소모하는 특정 스테이지 내부의 복잡한 수학 알고리즘이 전체 파이프라인을 기아 상태로 가로막게 된다. 네트워크 한 번 타고 오는 딜레이 시간에 비하면 인터페이스 변환 오버헤드는 우주의 먼지 수준만큼 미미하다.

(5) 결론

인터페이스 변환 구조를 사용해 파이프라인 모듈의 범용 재사용성을 극적으로 올리는 아키텍처적 전략은 실무에서 매우 훌륭하고 권장되는 모범 엔지니어링 설계이다.

하지만 만약 파이프라인 중간에 연산이 엄청나게 오래 걸리는 무거운 병목 스테이지가 딱 한 칸이라도 존재한다면, 상류에 있는 가볍고 영리한 제너레이터 고루틴들이 아무리 데이터를 뿜어대도 수송관 전체가 그 구역에 가로막혀 줄줄이 불로킹되는 심각한 속도 정체 현상이 터지게 된다. 이 치명적인 속도 병목을 물리치고 수천 개의 멀티코어 두뇌로 작업 분산 기적을 이뤄내는 최고급 동시성 기술인 ‘팬아웃/팬인’ 대형 연산 가속 패턴에 대해 알아보겠다.

[4-9] 팬아웃 / 팬인(Fan-out / Fan-in)

Go언어의 동시성 파이프라인 성능을 극적으로 끌어올리는 핵심 기술이자, 대규모 데이터 처리의 꽃인 팬아웃, 팬인 패턴에 대해 알아보겠다.

파이프라인을 구축하여 데이터가 여러 스테이지를 거쳐 아름답게 흐르고 가공되는 구조를 완성했다. 모든 것이 완벽해 보이지만, 실제 거대한 데이터를 흘려보내면, “도대체 왜 이렇게 느려터진 거지?”하고 답답해지는 순간이 찾아온다.

파이프라인 중 특정 스테이지가 복잡한 수학 연산이나 무거운 무작위 알고리즘을 수행하느라 유독 오랜 시간을 잡아먹기 때문이다. 이 무거운 스테이지 하나 때문에 상류에서 대기하던 가벼운 고루틴 일꾼들까지 수송관이 막혀 줄줄이 블로킹 상태에 빠지게 된다.

파이프라인 아키텍처의 위대한 속성 중 하나는, 스테이지들이 완전히 독립되어 서로 유연하게 조립되고 재사용할 수 있다는 점이다.

그렇다면 “가장 무거운 병목 구간 스테이지에만 여러 명의 고루틴 일꾼을 동시에 대거 투입(병렬화)해서 상류의 데이터를 나누어 튜브 연산 속도를 가속할 순 없을까?”라는 영리한 아이디어가 탄생한다. 이것이 바로 팬아웃, 팬인 패턴의 본질이다.

(1) 팬아웃과 팬인의 성립 조건

개념 정의는 다음과 같이 매우 명쾌하다.

- 팬아웃(Fan-Out): 상류 채널 수송관에서 쏟아지는 입력 데이터를 가공하기 위해, 동일한 스테이지 함수를 가진 독립된 고루틴 일꾼을 여러 개 동시에 띄워(분산) 작업을 분할 처리하는 과정이다.
- 팬인(Fan-In): 여러 개의 분산된 고루틴 일꾼들이 각자 연산을 마치고 내놓는 여러 개의 결과 채널 스트림들을 다시 단 하나의 깨끗한 통합 채널로 합쳐서(Multiplexing) 하류로 흘러 보내는 과정이다.

내가 설계한 파이프라인의 특정 스테이지에 이 팬아웃 기술을 적용할지 결정할 때는 다음 두 가지 조건이 모두 만족하는지 엄격히 따져봐야 한다.

팬아웃 적용을 위한 2대 필요충분조건

1. 해당 스테이지가 ‘이전 루프 단계에서 자기가 계산했던 과거의 상태나 값에 전혀 의존하지 않아야’한다. 즉, 완벽한 순서 독립성을 유지해야 한다.
2. 해당 스테이지의 내부 연산을 처리하는 데 걸리는 물리적인 시간이 유독 길고 무거워야 한다.

순서 독립성이 중요한 이유는, 여러 개의 고루틴 복사본들이 멀티코어 두뇌 위에서 동시에 날뛰며 연산을 수행할 때 그 어떤 일꾼이 먼저 연산을 끝내고 결과를 뱉어낼지 런타임 상에서 아무도 예측하거나 보장할 수 없기 때문이다. 값들이 무작위 순서로 튀어나와도 비즈니스 로직 상 안전한 영역이어야 이 패턴을 적용할 수 있다.

(2) 순차적 파이프라인의 속도 한계 증명

이 가속 패턴의 위력을 눈으로 직접 체감하기 위해, 먼저 팬아웃을 적용하지 않고 단 한 명의 일꾼 고루틴이 정직하게 순서대로 작동하는 무겁고 비효율적인 소수를 찾는 파이프라인을 구축해 보겠다.

상류에서 최고 50,000,000 이하의 거대한 난수를 무한 배출하면, primeFinder 스테이지가 넘겨받은 숫자를 그보다 작은 모든 숫자로 미련하게 하나씩 나누어보며 소수인지 판별하는 지독하게 느린 알고리즘이다.

생략 없는 전체 코드로 속도를 측정해 보겠다.

```
package main

import (
    "fmt"
    "math/rand"
    "time"
)

func main() {
    // 1. 무한 헬퍼 제너레이터 구현
    repeatFn := func(done <-chan interface{}, fn func() interface{}) <-chan interface{} {
        valueStream := make(chan interface{})
        go func() {
            defer close(valueStream)
            for {
                select {
                case <-done:
                    return
                case valueStream <- fn():
                }
            }
        }()
        return valueStream
    }

    // 2. 개수 제한 징수 커트 스테이지 구현
    take := func(done <-chan interface{}, valueStream <-chan interface{}, num int) <-chan interface{} {
        takeStream := make(chan interface{})
        go func() {
            defer close(takeStream)
            for i := 0; i < num; i++ {
                select {
                case <-done:
                    return
                case takeStream <- <-valueStream:
                }
            }
        }()
        return takeStream
    }
}
```


// 3. 빈 인터페이스를 정수형으로 캐스팅해 주는 스테이지 구현

```
toInt := func(done <-chan interface{}, valueStream <-chan interface{}) <-chan int {
    intStream := make(chan int)
    go func() {
        defer close(intStream)
        for v := range valueStream {
            select {
            case <-done:
                return
            case intStream <- v.(int):
            }
        }
    }()
    return intStream
}
```

// 4. [병목 구간] 무식하게 루프를 돌며 소수인지 판별하는 무거운 스테이지 구현

```
primeFinder := func(done <-chan interface{}, intStream <-chan int) <-chan
interface{} {
    primeStream := make(chan interface{})
    go func() {
        defer close(primeStream)
        for integer := range intStream {
            if integer <= 1 {
                continue
            }
            isPrime := true
            // 소수인지 비효율적으로 나누어 확인하여 인위적인 병목 지연

            for i := 2; i < integer; i++ {
                if integer%i == 0 {
                    isPrime = false
                    break
                }
            }
            if isPrime {
                select {
                case <-done:
                    return
                case primeStream <- integer:
                }
            }
        }
    }()
    return primeStream
}
```

유발

```
done := make(chan interface{})
defer close(done)
```

```
start := time.Now()
```

```

randFunc := func() interface{} { return rand.Intn(50000000) }
randIntStream := toInt(done, repeatFn(done, randFunc))

fmt.Println("Primes:")
// 무식한 단일 일꾼 primeFinder를 경유하여 소수 10개를 징수할 때까지 대기
for prime := range take(done, primeFinder(done, randIntStream), 10) {
    fmt.Printf("Wt%dWn", prime)
}
fmt.Printf("Search took: %vWn", time.Since(start))
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

Primes:
24941317
36122539
6410693
10128161
25511527
2107939
14004383
7190363
45931967
2393161
Search took: 23.437511647s

```

단, 10개의 소수를 추출하는데 무려 약23초라는 끔찍한 시간이 소요되었다.

일반적인 개발자라면 이 상황에서 알고리즘 책을 집어 들고 primeFinder 내부의 연산 공식 자체를 뜯어고치는 최적화를 먼저 떠올리겠지만, 이 단계가 인프라 API 호출이나 대용량 이미지 인코딩처럼 알고리즘 자체를 더 이상 단축시킬 수 없는 극한의 외부 병목 상황이라고 가정한다면, 우리는 구조적으로 일꾼의 개수를 찢는 팬아웃 기술을 도입해야만 한다.

(3) 초간단 팬아웃 구현 기법

Go 언어에서 특정 파이프라인 스테이지를 팬아웃(여러 개로 복사 분산)하는 과정은 허탈할 정도로 정말 쉽다.

자물쇠를 새로 채울 필요 없이, 그저 반복문을 돌리면서 동일한 스테이지 함수를 여러 번 동시에 실행하여 고루틴이 다중 복사본들을 띄워놓기만 하면 된다.

컴퓨터 하드웨어 CPU 코어 개수(runtime.NumCPU())만큼 독립된 채널 수송관 일꾼들을 대거 개설해보겠다.

```
// 기존의 단일 일꾼 가동 방식:
primeStream := primeFinder(done, randIntStream)

// 하드웨어 코어를 모두 점유하는 다중 팬아웃 가동 방식:
numFinders := runtime.NumCPU() // 8코어 장비라면 8이 반환됨
finders := make([]<-chan interface{}, numFinders)
for i := 0; i < numFinders; i++ {
    // 8명의 독립된 고루틴 소수 판별 일꾼들이 하나의 randIntStream 수송관에 입을 대고
    // 들어오는 난수 데이터를 비동기식으로 뺏어 가며 연산을 나누어 처리합니다.
    finders[i] = primeFinder(done, randIntStream)
}
```

이제 상류에서 쏘아 올린 난수들은 노는 컴퓨터 두뇌 하나 없이 8명의 일꾼에게 골고루 흩어져 빛의 속도로 난도질당하기 시작한다.

하지만 여기서 새로운 아키텍처 문제가 발생한다.
일꾼을 8개로 쪼개 놓았더니 연산 결과를 내뿜는 결과 채널 수송관 역시 finders[0]부터 finders[7]까지 총 8개로 분열되어 버린 것이다.
최종 하류에 있는 소비자인 for range 구문은 오직 단 하나의 깨끗한 단일 채널을 통해서만 데이터를 수집할 수 있는데 말이다.

이 분열된 8개의 데이터 물줄기를 단 하나의 통로로 안전하게 합병해 주는 기술이 바로 다음 단계인 팬인(Fan-In)패턴이다.

(4) 다중 채널을 하나로 묶는 팬인(Fan-in) 알고리즘 구현

여러 개의 가변 인자 슬라이스를 입력받아, 내부 다중 멀티플렉서(Multiplexer) 고루틴들을 가동시켜 하나의 최종 채널 스트림으로 녹여내는 정석적인 fanIn 함수 전체 소스코드이다.

```
package main

import (
    "sync"
)

// 여러 개의 채널을 받아 단 하나의 복합 수신 채널로 합쳐주는 팬인 함수 정의
func fanIn(done <-chan interface{}, channels ...<-chan interface{}) <-chan interface{} {
    var wg sync.WaitGroup
    multiplexedStream := make(chan interface{})

    // 주입된 채널 하나를 전담 마크하여 값을 끄집어내 공용 통합 채널에 전달하는 멀티플렉서 일꾼 정의
    multiplex := func(c <-chan interface{}) {
        defer wg.Done()
        for i := range c {
            select {
            case <-done:
                return
            case multiplexedStream <- i: // 통합 채널 파이프라인으로 데이터 토스
            }
        }
    }
}
```

```

// 핵심 조율: 넘겨받은 모든 분열된 채널의 개수만큼 WaitGroup 카운터를 올립니다.
wg.Add(len(channels))
for _, c := range channels {
    // 각 채널당 독립적인 고루틴 멀티플렉서 일꾼을 1:1로 매칭하여 가동 (팬인 실행)
    go multiplex(c)
}

// 모든 채널로부터 가방이 텅 비어 수집이 끝나면, 통합 채널 수송관을 안전하게 닫고 퇴근하는 감시자 고루틴
go func() {
    wg.Wait()
    close(multiplexedStream)
}()

return multiplexedStream
}

```

동작 아키텍처 원리는 매우 명쾌하다.

8개의 채널이 들어오면 8개의 가벼운 multiplex 고루틴을 띄우고, 각자 맡은 채널 방지턱에 서서 데이터가 나오는 대로 낚아채 하나의 공용 통합 채널(multiplexedStream)로 사정없이 털어 넣었다.

이 8명의 일꾼이 각자의 채널이 닫혀서 전원 임무를 종료(wg.Done())하면, 뒤편에서 대기하던 감시자 고루틴의 wg.Wait() 자물쇠가 풀리면서 통합 채널 수송관을 안전하게 close하고 파이프라인 정리를 깔끔하게 매듭짓는 마스터 동기화 조율 구조이다.

(5) 팬아웃, 팬인 결합 가속 파이프라인 전체 소스코드

이제 병목 구간에 8개의 일꾼을 분산 투입하고(Fan-Out), 분열된 수송관 물줄기를 하나로 통합하는(Fan-In) 기적의 대용량 동시성 가속 파이프라인의 생략없는 전체 코드를 보자.

```
package main
```

```

import (
    "fmt"
    "math/rand"
    "runtime"
    "sync"
    "time"
)

func main() {
    repeatFn := func(done <-chan interface{}, fn func() interface{}) <-chan interface{} {
        valueStream := make(chan interface{})
        go func() {
            defer close(valueStream)
            for {
                select {
                case <-done:
                    return
            }
        }
    }
}

```

```

        case valueStream <- fn():
        }
    }
}()
return valueStream
}

take := func(done <-chan interface{}, valueStream <-chan interface{}, num int) <-
chan interface{} {
    takeStream := make(chan interface{})
    go func() {
        defer close(takeStream)
        for i := 0; i < num; i++ {
            select {
            case <-done:
                return
            case takeStream <- <-valueStream:
            }
        }
    }()
    return takeStream
}

toInt := func(done <-chan interface{}, valueStream <-chan interface{}) <-chan int {
    intStream := make(chan int)
    go func() {
        defer close(intStream)
        for v := range valueStream {
            select {
            case <-done:
                return
            case intStream <- v.(int):
            }
        }
    }()
    return intStream
}

primeFinder := func(done <-chan interface{}, intStream <-chan int) <-chan
interface{} {
    primeStream := make(chan interface{})
    go func() {
        defer close(primeStream)
        for integer := range intStream {
            if integer <= 1 {
                continue
            }
            isPrime := true
            for i := 2; i < integer; i++ {
                if integer%i == 0 {
                    isPrime = false
                    break
                }
            }
            if isPrime {
                primeStream <- integer
            }
        }
    }()
    return primeStream
}

```

```

    }
    }
    if isPrime {
        select {
            case <-done:
                return
            case primeStream <- integer:
        }
    }
}()
return primeStream
}

fanIn := func(done <-chan interface{}, channels ...<-chan interface{}) <-chan
interface{} {
    var wg sync.WaitGroup
    multiplexedStream := make(chan interface{})

    multiplex := func(c <-chan interface{}) {
        defer wg.Done()
        for i := range c {
            select {
                case <-done:
                    return
                case multiplexedStream <- i:
            }
        }
    }

    wg.Add(len(channels))
    for _, c := range channels {
        go multiplex(c)
    }

    go func() {
        wg.Wait()
        close(multiplexedStream)
    }()

    return multiplexedStream
}

done := make(chan interface{})
defer close(done)

start := time.Now()
randFunc := func() interface{} { return rand.Intn(500000000) }
randIntStream := toInt(done, repeatFn(done, randFunc))

// [팬아웃 가동 지점] 컴퓨터 코어 개수만큼 소수 판별 일꾼을 대거 생성 및 복사 분산
numFinders := runtime.NumCPU()

```

```

    fmt.Printf("Spinning up %d prime finders.\n", numFinders)
    finders := make([]chan interface{}, numFinders)

    fmt.Println("Primes:")
    for i := 0; i < numFinders; i++ {
        finders[i] = primeFinder(done, randIntStream)
    }

    // [팬인 가동 지점] 분열된 일꾼 채널들을 다시 단 하나의 채널 스트림으로 병합하여 소
    비
    for prime := range take(done, fanIn(done, finders...), 10) {
        fmt.Printf("Wt%d\n", prime)
    }
    fmt.Printf("Search took: %v\n", time.Since(start))
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

Spinning up 8 prime finders.
Primes:
    6410693
    24941317
    10128161
    36122539
    25511527
    2107939
    14004383
    7190363
    2393161
    45931967
Search took: 5.438491216s

```

(6) 결론

연산 가속 결과를 비교해보자. 처음 단일 일꾼으로 정직하게 순서대로 연산했을 때 약 23초가 걸렸던 대형 병목 파이프라인 시스템이, 팬아웃/팬인 다중 결합 패턴을 도입했더니 단 약 5초만에 완벽하게 처리를 완수해 냈다.

성능 최적화 지표

- 기존 순차 파이프라인 실행 속도: 23.43초
- 팬아웃, 팬인 병렬 파이프라인 실행 속도: 5.43초
- 결론: 전체 프로그램 구조를 무리하게 바꾸지 않고 오직 동시성 조립 라인 최적화만으로 약 78%의 전체 실행 시간 단축을 기적처럼 달성했다.

다만 다시 한번 기억해야 할 사실은, 이 단순한 기본형 팬아웃/팬인 아키텍처는 결과물 데이터가 하루 소비자에게 도달하는 출력 순서가 비즈니스 상 전혀 중요하지 않을 때만 성립한다는 점이다. 8명의 일꾼이 난수를 무작위로 채 가고 먼저 끝난 순서대로 팬인 채널에 집어던졌기 때문에 상류에서 들어간 원본 데이터의 선입선출(FIFO) 순서는 완벽하게 파괴당 흩어지게 된다.

만약 가속 연산을 처리하면서도 데이터의 입력 순서를 칼같이 복구하여 유지해야 하는 더 고차원적인 실무 설계 기법은 책의 다음 장에서 흥미진진하게 이어진다.
여러분은 방금 Go 언어가 자랑하는 분산 대량 연산 가속 패턴의 정수를 완벽하게 정복했다.

[4-10] tee-channel 패턴

Go 언어의 고급 동시성 패턴 중, 하나의 채널로 들어오는 데이터 물줄기를 완벽하게 복제하여 두 개의 독립된 서로 다른 시스템으로 똑같이 분배해주는 패턴을 알아보겠다.

실무에서 서비스를 개발하다 보면, 하나의 채널로 인입되는 데이터 스트림을 서로 다른 두 개의 코드 영역으로 동시에 짜주어야 하는 요구사항이 자주 발생한다.

예를 들어 ‘사용자의 명령어’가 들어오는 단일 채널이 있을 때, 이 명령어를 가져다가 실제로 비즈니스 로직을 수행하는 프로세스에게도 보내야 하고, 동시에 보안 감사나 모니터링을 위해 데이터베이스에 로그를 남기는 프로세스에게도 똑같이 전달해야 하는 상황이다.

Unix/Linux 계열 운영체제의 유명한 명령줄 도구인 tee 명령어에서 이름을 딴 이 패턴은, 입력 채널 하나를 넘겨 받아 완벽하게 동일한 자원이 흘러나오는 두 개의 독립된 채널을 쌍둥이처럼 반환해준다.

(1) tee-channel 패턴의 전체 소스코드 구현

상류의 데이터 파이프라인 수송관에서 값을 받아 하류의 두 갈래 채널망으로 공평하게 복제 전달하는 tee 함수의 생략 없는 전체 코드를 확인해보겠다.

이 코드에는 orDone 이라는 채널 안전 순회 헬퍼 함수가 결합되어 작동한다.

```
package main

import (
    "fmt"
)

// 헬퍼 함수: 상류 채널을 순회할 때 done 취소 신호가 오면 즉시 안전하게 루프를 깨주는 도구
func orDone(done <-chan interface{}, c <-chan interface{}) <-chan interface{} {
    valStream := make(chan interface{})
    go func() {
        defer close(valStream)
        for {
            select {
            case <-done:
                return
            case v, ok := <-c:
                if ok == false {
                    return
                }
                select {
                case valStream <- v:
                case <-done:
                }
            }
        }
    }()
    return valStream
}

func main() {
```

// 1. 입력 채널(in) 하나를 받아 복제된 두 개의 수신 채널(out1, out2)을 반환하는 tee 함수 정의

```
tee := func(
    done <-chan interface{},
    in <-chan interface{},
) (<-chan interface{}, <-chan interface{}) {
    out1 := make(chan interface{})
    out2 := make(chan interface{})
```

```
    go func() {
        defer close(out1)
        defer close(out2)
```

// orDone 방어벽을 경유하여 입력 채널에서 값을 하나씩 안전하게 꺼내옵니다.

```
    for val := range orDone(done, in) {
        // 핵심 테크닉: 외부 변수 오염을 막기 위해 로컬 영역에서 out1,
        out2 변수를 쉐도잉(Shadow)합니다.
```

```
        var out1, out2 = out1, out2
```

// 하나의 select 구문 내에서 두 채널이 서로를 가로막지 (Blocking) 않고

// 두 채널 모두에게 공평하게 데이터가 1번씩 가 전송되도록 정확히 2번의 루프를 강제 가동합니다.

```
    for i := 0; i < 2; i++ {
        select {
            case <-done:
                return
            case out1 <- val:
```

// out1 채널에 데이터 전송 성공 시, 해당 복사본 변수를 nil로 잡아 끼워버립니다.

// Go 언어에서 nil 채널에 데이터를 넣으려는 행위는 영원히 대기(Block) 상태에 빠지므로,

// 다음 2번째 루프 차례 때 select 문이 이미 성공한 out1 케이스를 무시하고

// 아직 전송되지 않은 out2 케이스 쪽으로 자연스럽게 시선을 돌리게 만드는 고도의 기술입니다.

```
        out1 = nil
        case out2 <- val:
```

// out2 채널에 데이터 전송 성공 시, 역시 해당 복사본 변수를 nil로 지워버립니다.

```
        out2 = nil
```

```
    }
```

```
    }
```

```
    }0
```

```
    return out1, out2
```

```
}
```

// 2. 가상의 연속 데이터를 배출하는 간단한 제너레이터와 정수 스테이지 임시 구현

```
repeat := func(done <-chan interface{}, values ...interface{}) <-chan interface{} {
```

```

        valueStream := make(chan interface{})
        go func() {
            defer close(valueStream)
            for {
                for _, v := range values {
                    select {
                        case <-done:
                            return
                        case valueStream <- v:
                    }
                }
            }
        }()
        return valueStream
    }

    take := func(done <-chan interface{}, valueStream <-chan interface{}, num int) <-
chan interface{} {
        takeStream := make(chan interface{})
        go func() {
            defer close(takeStream)
            for i := 0; i < num; i++ {
                select {
                    case <-done:
                        return
                    case takeStream <- <-valueStream:
                }
            }
        }()
        return takeStream
    }

    done := make(chan interface{})
    defer close(done)

    // 3. 1, 2, 1, 2 패턴 데이터를 총 4개 징수한 채널 수송관을 tee 함수에 주입하여 분할 실행
    out1, out2 := tee(done, take(done, repeat(done, 1, 2), 4))

    // 두 갈래로 나뉘어 뿔어져 나오는 쌍둥이 채널 데이터를 동시에 받아 출력 검증
    for val1 := range out1 {
        fmt.Printf("out1: %v, out2: %v\n", val1, <-out2)
    }
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

out1: 1, out2: 1
out1: 2, out2: 2
out1: 1, out2: 1
out1: 2, out2: 2

```

(2) 섀도잉과 Nil 채널을 활용한 비블로킹 분배 법칙

출력 결과를 보시면 단 하나의 입력 소스로부터 출발한 데이터들이 out1과 out2라는 완벽하게 분리된 채널망 양쪽 모두로 누락 없이 공평하게 복제 전달된 것을 확인할 수 있다.

이 패턴의 핵심 진가는 for i:= 0; i < 2; i++ 루프와 select 문 내부에 숨겨진 ‘Nil 채널 트릭’ 아키텍처에 있다.

일반적으로 두 개의 채널에 순서대로 값을 전송하려고 하면 다음과 같은 고질적인 동시성 병목이 터지게 된다.

```
// 나쁜 직렬 전송 예시:
```

```
out1 <- val // 만약 out1 시스템이 바빠서 데이터를 안 받아 가고 막혀있다면?
```

```
out2 <- val // 아래쪽에 있는 out2 시스템은 한가하고 준비가 되었는데도 일 시작조차 못 하고  
대기해야 함
```

이 직렬 전송의 문제를 타파하기 위해,
tee 패턴은 단 하나의 select 구문을 사용해 두 채널의 송신 기회를 동등한 선상에 올려놓았다.
그리고 둘 중 어느 한쪽 채널이라도 먼저 데이터를 낚아채서 전송에 성공하면,
로컬 변수에 보관 중이던 채널 복사본의 값을 즉시 nil로 갈아치운다.

우리가 3장의 채널 치트키 지도를 공부할 때 “초기화되지 않은 nil 채널에 값을 넣으려고 하면 영원한 대기 상태에 빠진다”고 배웠던 스펙 기억하나?
이 무서운 스펙을 역으로 이용한 최적화 기술이다.

첫 번째 루프에서 out1 전송이 먼저 성공하여 변수 값이 nil이 되어버리면,
곧바로 도는 두 번째 select 루프 차례 때 out1 케이스는 평생 대기 구역으로 넘어가 버리므로,
Go 런타임이 select 엔진은 안전하게 아직 전송이 완료되지 않은 나머지 out2 케이스 케이스 쪽을
꼭 집어 선택해 연산을 무사히 끝마치게 된다. 자물쇠를 단 한개도 채우지 않고 완벽한 비동기식
병렬 데이터 다중 토스를 이뤄낸 예술적인 구조이다.

(3) 실무 설계 시 주의해야 할 결합도 특성

이 위대한 최적화 구조를 실무 대형 아키텍처 시스템에 도입할 때,
한 가지 명심해 두어야 할 구조적 결합도 특징이 있다.

방금 본 것처럼, out1과 out2 채널로의 데이터 복제 전송은 내부적으로 철저하게 단단히 결합되어 움직인다.

상류 파이프라인에서 들어오는 데이터 in 변수의 다음 루프 순회 연산은,
out1과 out2라는 두 마리의 토끼 모두에게 데이터가 완벽하게 전달이 완료되기 전까지 절대로
다음 줄로 넘어가지 못하고 정체된다.

즉, 만약 로그를 수집하는 out2 고루틴 시스템쪽에 심각한 메모리 과부하가 걸려 채널 창고를
전혀 비워내지 못하고 멈춰 서 버린다면, 비즈니스 연산을 아주 생생하고 빠르게 수행하던
out1 고루틴 시스템 쪽까지 억울하게 수송관이 막혀 연쇄적으로 동작이 마비될 수 있다.

물론 이렇게 파이프라인 하류에서 발생하는 처리량 오버헤드와 소비 속도를 균일하게
마사지하는 책임은 tee 명령어 자체의 관심사가 아니라, 데이터를 가져다 쓰는 하류 소비자
시스템 측에서 풀어나가야 할 영역(예: 버퍼 채널 도입이나 팬아웃 가속 결합 등)이 맞다.

하지만 수송관 전체의 생명줄이 연결되어 있다는 특성은 설계 단계에서 늘 인지하고 있어야 한다.

(4) 결론

이 tee-channel 패턴은 시스템의 핵심 비즈니스 로직 코드와 로깅, 감사, 데이터 복제, 모니터링 같은 부가적인 인프라 스트럭처 로직 코드를 코드 단에서 서로의 존재를 전혀 모르게 완벽한 격리 상태를 유지하면서도, 시스템 통합 교차점에서 부드럽게 결합할 수 있도록 도와주는 위대한 동시성 벽돌이다.

채널이 가진 원초적인 기하학적 규칙들만으로 이런 우아한 아키텍처를 뽑아낼 수 있다는 점이 바로 전 세계 수많은 설계자가 Go언어의 동시성에 매료되는 진짜 이유이다.

[4-11] bridge-channel

Go언어의 대규모 동시성 파이프라인 아키텍처 중에서, 차례대로 인입되는 여러 채널의 데이터 스트림을 단 하나의 깨끗한 단일 채널로 매끄럽게 이어붙여주는 bridge-channel 패턴에 대해 알아보겠다.

실무에서 동시성 시스템을 설계하다 보면 다음과 같은 매우 독특한 묘한 형태의 채널을 다루어야 할 때가 있다.

```
<-chan <-chan interface{}
```

이것은 채널에 값을 꺼냈더니 데이터가 나오는 것이 아니라, 또 다른 채널(<-chan interface{})이 튀어나오는 ‘채널을 담는 채널’ 구조이다.

앞서 배운 팬아웃/팬인 패턴이나 or-channel 패턴이 ‘동시에 존재하는 여러 채널’의 물줄기를 하나로 합치는 자원 병합이었다면,

이 bridge 패턴은 ‘시간 순서대로 차례대로 밀려오는 복수의 채널 스트림’을 다룬다. 서로 다른 소스로부터 순서가 보장된 쓰기가 일어날 때 이 구조가 유용하게 쓰인다.

가장 대표적인 실무 사례는 수명이 불연속적으로 끊겼다가 간헐적 파이프라인 스테이지를 다룰때이다. 4장의 구속 원칙에 따라 “채널의 소유권은 해당 채널에 값을 쓰는 고루틴이 가진다”는 규칙을 철저히 지키면, 백그라운드 고루틴 일꾼이 에러 등으로 인해 죽었다가 새로운 고루틴으로 재시작될 때마다 매번 완전히 새로운 채널 인스턴스가 생성된다. 이로 인해 필연적으로 채널들이 순서대로 나열되는 채널의 시퀀스가 발생하게 된다.

이 데이터를 최종적으로 받아다 쓰는 소비자 관점에서는, 이 자원이 영켜있는 채널의 채널인지 뭔지 내부 사정을 알고 싶지 않다. 그저 막힘없이 데이터가 한 줄로 뿜어져 나오는 단일 채널만 바라보고 싶을 뿐이다.

이처럼 복잡한 채널의 채널 구조를 일반적인 단순 단일 채널로 해체하여 소비자에게 파사드(Façade) 대문 역할을 해주는 기술을 채널 브리징(Bridging the channels)이라고 부른다.

(1) bridge-channel 패턴의 전체 소스코드 구현

상류에서 끝없이 밀려오는 채널 뭉치들을 풀어서 하류의 소비자에게 단 하나의 unbroken stream(끊이지 않는 물줄기)으로 이어 붙여 전달하는 bridge 함수의 생략 없는 전체 코드를 확인해보겠다.

이 코드 역시 채널 안전 순회용 헬퍼 함수인 orDone이 단단하게 결합되어 움직인다.

```
package main

import (
    "fmt"
)

// 헬퍼 함수: 상류 채널을 순회할 때 done 취소 신호가 오면 즉시 안전하게 루프를 깨주는 도구
func orDone(done <-chan interface{}, c <-chan interface{}) <-chan interface{} {
    valStream := make(chan interface{})
    go func() {
        defer close(valStream)
        for {
            select {
            case <-done:
                return
            case v, ok := <-c:
                if ok == false {
                    return
                }
                select {
                case valStream <- v:
                case <-done:
                }
            }
        }
    }()
    return valStream
}

func main() {
    // 1. 채널을 담은 채널Stream을 해체하여 단일 수신 채널로 이어 붙여주는 bridge 함수 정의
    bridge := func(
        done <-chan interface{},
        chanStream <-chan <-chan interface{},
    ) <-chan interface{} {
        // bridge 함수가 최종적으로 모든 데이터 값을 하나로 모아 반환할 통합 수송관
        valStream := make(chan interface{})

        go func() {
            defer close(valStream)
            for {
                var stream <-chan interface{}
            }
        }()
    }
}
```

을 하나씩 꺼내옵니다.

종 퇴근

닫혀서 텅 비어

통합 채널로 토스합니다.

위로 올라간 뒤

한히 이어 나갑니다.

```
// 바깥쪽 루프: 큰 물줄기 채널Stream으로부터 내부 자식 채널들
```

```
select {  
  case maybeStream, ok := <-chanStream:  
    if ok == false { // 더 이상 들어올 채널 뭉치가 없으면 최  
      return  
    }  
    stream = maybeStream  
  case <-done:  
    return  
}
```

```
// 안쪽 루프: 방금 대기조에서 꺼내온 개별 자식 채널이 완전히
```

```
// 바닥을 드러낼 때까지 내부 데이터 값(val)들을 끄집어내 공용
```

```
// 현재 자식 채널이 다 비어서 close되면, 안쪽 루프를 탈출하여
```

```
// 다음 차례의 자식 채널을 대기조에서 새로 뽑아와 이 작업을 무
```

```
for val := range orDone(done, stream) {  
  select {  
    case valStream <- val:  
    case <-done:  
  }  
}
```

```
  }  
}
```

```
return valStream
```

```
}
```

터 구현 // 2. 실행 검증을 위해 10개의 독립된 채널을 순차적으로 생성해 뽑어내는 가상 제너레이터

```
genVals := func() <-chan <-chan interface{} {  
  chanStream := make(chan (<-chan interface{}))  
  go func() {
```

연속 생산

```
    defer close(chanStream)  
    for i := 0; i < 10; i++ {  
      // 데이터를 딱 1개만 넣고 즉시 close 시켜버리는 단명 채널들을
```

밀어 넣음

```
      stream := make(chan interface{}, 1)  
      stream <- i  
      close(stream)
```

```
// 채널을 담는 채널 통로 속으로 이 자식 채널 인스턴스 자체를
```

```
chanStream <- stream
```

```
  }  
}
```

```
return chanStream
```



```

    }

    // 3. 복잡한 채널의 채널 소스를 bridge 함수에 통째로 통과시켜 단일 투명 채널로 정제
함 // 메인 함수(소비자)는 오직 단 한 줄의 단순한 for range 구문만으로 원형의 논리 데이터
를 깔끔하게 수집합니다.
    for v := range bridge(nil, genVals()) {
        fmt.Printf("%v ", v)
    }
    fmt.Println()
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
0 1 2 3 4 5 6 7 8 9
```

(2) 이중 루프 제어를 통한 Seamless(심리스) Stream 아키텍처의 원리(경계 없는 스트림)

출력 결과를 보면 상류에서 각기 다른 타이밍에 생성되고 쪼개져서 파편화되어 들어온 10개의 독립된 채널 자원들이, 최종 소비자 레벨에서는 단 한 자릿수도 튀기거나 끊기지 않고 0부터 9까지 물 흐르듯 단 한 줄의 심리스한 통합 데이터 스트림으로 이쁘게 출력된 것을 볼 수 있다.₩

이 마법같은 파사드 제어가 가능한 비결은 bridge 함수 내부에 정교하게 설계된 ‘이중 루프 컨베이어 벨트 구조’ 덕분이다.

- 바깥쪽 select 루프

거대한 채널의 채널 저장소(chanStream)를 삼엄하게 감시하며 대기하다가, 새로운 고루틴 일꾼이나 모듈이 “저 새로 태어났어요!”하고 새로운 채널 인스턴스를 던져주면 이를 획 낚아챈다.

- 안쪽 for range 루프

방금 전달받은 자식 채널의 입구에 바짝 붙어서 내부에 들어있는 데이터 값들을 남김없이 쏙쏙 빨아내어 최종 공용 채널(valStream)로 옮겨 싣는다.

그러다가 해당 자식 채널의 데이터가 완전히 바닥나서 상류 생산자가 그 채널을 닫아버리면(close), 안쪽 루프는 즉시 푹푹하게 소멸되면서 다시 바깥쪽 스케줄러 루프에게 제어권을 토스한다. 그리고 곧바로 대기열에서 있던 다음 차례의 자식 채널을 새로 공급받아 다시 내부 데이터를 빨아내기 시작한다.

이 연속적인 교대근무 연산 덕분에 하류의 소비자는 상류에서 채널이 터지고 죽고 새로 복구되는 구체적인 하드웨어 아키텍처 대혼란 상황을 전혀 눈치채지 못한채, 맑고 투명하게 정제된 하나의 단일 채널만 바라보며 자신의 순수한 비즈니스 루프 논리에만 100% 집중할 수 있게 되는 것이, 관심사의 완벽한 격리이자 분리이다.

1. 내부(orDone)가 먼저 done 감지
2. 내부 range 종료
3. 안쪽 for문 종료
4. 다시 바깥 for문으로 올라감
5. 바깥 select에서도 done 감지
6. 최종 return

(3) 결론

이 bridge-channel 패턴은 뒤이어 배울 시스템의 자가 치유 아키텍처인 ‘건강하지 못한 고루틴의 에러 복구 자가 치유’ 엔진을 구현할 때 모든 동시성 제어 라인을 하나로 결합하는 거대한 중심 뼈대로 아주 중요하게 사용된다.

동시성 시스템의 규모가 커져서 수많은 일꾼 고루틴들이 수시로, 죽고 새 채널을 파며 부활하는 다이내믹한 클라우드 웹 스케일 환경을 통제해야 한다면, 이 우아한 채널 브리징 기법을 결합하여 내부의 복잡성을 감추고 최고 수준의 정돈된 유지보수성을 확보하자.

[4-13] Queuing

Go 언어의 동시성 파이프라인 설계에서 시스템의 유연성과 안정성을 조율하는 핵심 기술인 큐잉(Queuing, 대기열 도입) 패턴에 대해 알아보겠다.

파이프라인이 다음 작업을 받을 준비가 아직 되지 않았더라도, 상류 스테이지가 완료한 작업을 임시 보관소에 저장하여 선제적으로 다음 일감을 수락하는 프로세스를 큐잉이라고 한다. 3장에서 배웠던 ‘버퍼를 가진 채널’이 바로 고루틴 세계의 가장 대표적인 임시 큐이다.

큐잉은 매우 유용한 기술이지만, 프로그램 최적화 단계에서 가장 마지막에 도입해야 하는 양날의 검이다. 준비되지 않은 상태에서 버퍼를 조기 도입하면 데드락이나 라이브락같은 치명적인 동시성 동기화 버그들이 눈앞에서 일시적으로 감춰지기 때문에, 오히려 완벽한 정답코드로 수렴하는 데 방해가 되기 때문이다.

(1) 큐를 도입해도 프로그램 전체 속도는 빨라지지 않는다.

많은 개발자가 저지르는 가장 흔한 성능 튜닝의 실수가 바로 “시스템이 느리니까 중간에 큐(버퍼)를 채워 넣어서 속도를 올려야겠다”라는 생각이다.

결론부터 말하자면, 큐를 도입한다고 해서 프로그램의 전체 실행 시간(total runtime)이 단 1초도 줄어들지는 않는다. 큐는 프로그램의 연산 처리 속도를 물리적으로 가속하는 도구가 아니라, 프로그램이 막혔을 때 대기하는 동작 방식을 다르게 바꾸어 줄 뿐이다.

이 진실을 수학적으로 증명하기 위해, 1초동안 잠드는 가벼운 스테이지(short)와 4초 동안 잠드는 무거운 스테이지(long)가 직렬로 연결된 파이프라인 시뮬레이션을 돌려보겠다.

버퍼가 없는 순차적 파이프라인 타임라인

시간(t)	루프차레(i)	무거운 스테이지(Long)	가벼운 스테이지(Short)
0s	0	0s 남음	1s 남음
1s	1	4s 남음	0s 남음
2s	2	3s 남음	(막힘, Blocked)
3s	3	2s 남음	(막힘, Blocked)
4s	4	1s 남음	(막힘, Blocked)
5s	1	4s 남음	1s 남음
...			
13s	3 (최종 종료)		

버퍼가 없을 때 이 파이프라인 3개의 데이터를 처리하는 데 걸리는 총시간은 정확히 13초이다. 이때 주목할 점은, 가벼운 스테이지(Short)가 상류에서 일을 끝내놓고도 하류의 무거운 일꾼이 자리를 비워주지 않아 무려 9초동안이나 꼼짝 못하고 막힌 상태로 서 있었다는 사실이다.

중간에 2칸짜리 버퍼 큐를 도입한 파이프라인 타임라인

이제 가벼운 일꾼과 무거운 일꾼 사이에 값을 2개까지 임시 보관할 수 있는 버퍼 대기열 창고 (buffer := make(chan int, 2))를 타임라인 중간에 뚫어보겠다.

시간(t)	루프차례(i)	무거운 스테이지(Long)	버퍼 창고 상태	가벼운 스테이지(Short)
0s	0	0s 남음	0/2 칸 점유	1s 남음
1s	1	4s 남음	0/2 칸 점유	1s 남음
2s	2	3s 남음	1/2 칸 점유	1s 남음
3s	3	2s 남음	2/2 칸 점유	(창고 가득 참, close)
4s	4	1s 남음	2/2 칸 점유	-
5s	1	4s 남음	1/2 칸 점유	-
...				
13s	3 (최종 종료)			

결과 지표를 확인해보자. 전체 파이프라인이 종료되는 데 걸린 시간은 버퍼가 없을 때와 완벽하게 동일한 13초이다. 전체 속도는 전혀 빨라지지 않았다.

하지만 엄청난 반전이 있다. 가벼운 스테이지(Short)의 동작을 보자.

이전에는 하류 일꾼 때문에 9초 동안 락이 걸려 묶여 있었는데, 버퍼 창고가 일감을 대신 맡아준 덕분에 단 3초만에 자신의 임무를 깔끔하게 완수하고 조기 퇴근에 성공했다.

대기 블로킹 시간이 3분의 1로 대폭 감축된 것이다.

(2) 대기열(Queue)의 진짜 유용성과 실무적 가치

“전체 실행 시간이 똑같이 13초라면, 상류 일꾼이 일찍 끝나는 게 도대체 시스템에 무슨 도움이 되나?”

인터넷 연결 요청을 직접 수락하는 실제 프로덕션 웹 서버의 아키텍처 파이프라인 구조를 떠올리면 이 질문에 대한 해답이 명확해진다.

```
// 실무형 네트워크 연결 수락 파이프라인 예시
pipeline := processRequest(done, acceptConnection(done, httpHandler))
```

이 시스템은 사용자가 접속을 끊기 전까지 무한 가동된다.

만약 유저의 비즈니스 요청을 복잡하게 처리하는 processRequest 스테이지에 일시적인 대형 병목이 발생했다고 가정해보자.

중간에 버퍼 대기열 큐가 없다면, 하류의 병목 신호가 상류로 그대로 전파되어 새로운 유저의 네트워크 노크를 받아 주는 acceptConnection 스테이지까지 그 자리에서 락이 걸려 굳어버리게 된다. 이렇게 되면 새로 들어오려는 무고한 유저들은 접속 요청 자체가 아예 거절당하는 서비스 거부(Connection Timeout / Denial of service) 대참사를 맞이하게 된다.

여기에 대기열 큐를 뚫어놓으면, 비록 하류 연산이 밀려서 최종 응답 리턴 속도는 느려질지언정 (Lag 발생), 상류의 안내원 고루틴은 막힘없이 계속 깨어나 밀려드는 유저들의 연결 요청을 안전하게 수락하여 창고에 보관해 둘 수 있게 된다.

즉, 유저들에게 “잠시만 대기해 주세요”라는 안내판을 띄우며 서비스 다운없이 안정적으로 트래픽을 받아낼 수 있게 되는 것이다.

큐잉(Queuing) 패턴의 진짜 목적

큐의 진정한 가치는 스테이지 간의 연산 실행 성능을 독립적으로 격리하는 완벽한 디커플링(결합도 해제)에 있다. 상류 일꾼이 하류 일꾼의 컨디션 난조에 영향을 받지 않고 본연의 임무를 완수할 수 있도록 자유를 주는 것이다.

(3) 효율적인 배치 처리를 증명하는 벤치마크(Chunking 성능 검증)

이렇게 스테이지 간 결합도를 해제하는 목적 외에, “데이터를 임시 대기열에 모아두었다가 한꺼번에 묶어서 털어 넣는 배치(Batching) 혹은 청킹(Chunking) 연산”을 수행할 때 큐가 물리적인 프로그램 속도 자체를 결합적으로 끌어올려 주기도 한다.

가장 대표적인 사례가 바로 메모리에 데이터를 모았다가 디스크 파일에 한 방에 내려 쓰는 Go 언어의 내장 기본 라이브러리인 bufio 패키지이다. 디스크 하드웨어에 한 바이트씩 10만 번 저장하는 방식과, 큐 창고에 모았다가 덩어리로 한 번에 내려치는 방식의 속도 격차를 생략 없는 전체 벤치마크 코드로 증명해 보겠다.

```
package main

import (
    "bufio"
    "fmt"
    "io"
    "io/ioutil"
    "log"
    "os"
    "testing"
)

// 헬퍼 스테이지 1: 무한 반복 배출
func repeat(done <-chan interface{}, values ...interface{}) <-chan interface{} {
    valueStream := make(chan interface{})
    go func() {
        defer close(valueStream)
        for {
            for _, v := range values {
                select {
                case <-done:
                    return
                case valueStream <- v:
                }
            }
        }
    }()
    return valueStream
}

// 헬퍼 스테이지 2: 개수 커트
func take(done <-chan interface{}, valueStream <-chan interface{}, num int) <-chan interface{} {
    takeStream := make(chan interface{})
    go func() {
        defer close(takeStream)
        for i := 0; i < num; i++ {
            select {
            case <-done:
                return
            case takeStream <- <valueStream:
            }
        }
    }()
    return takeStream
}
```

```

        for i := 0; i < num; i++ {
            select {
            case <-done:
                return
            case takeStream <- <-valueStream:
            }
        }
    }()
    return takeStream
}

func tmpFileOrFatal() *os.File {
    file, err := os.CreateTemp("", "tmp")
    if err != nil {
        log.Fatalf("error: %v", err)
    }
    return file
}

func performWrite(b *testing.B, writer io.Writer) {
    done := make(chan interface{})
    defer close(done)
    b.ResetTimer()
    // 파이프라인으로부터 바이트 데이터를 하나씩 꺼내 파일 쓰기 수행
    for bt := range take(done, repeat(done, byte(0)), b.N) {
        writer.Write([]byte{bt.(byte)})
    }
}

// 측정 A: 대기열 큐 없이 매번 디스크 물리 파일에 직통으로 한 바이트씩 쓰는 방식
func BenchmarkUnbufferedWrite(b *testing.B) {
    performWrite(b, tmpFileOrFatal())
}

// 측정 B: bufio 내부 큐 창고에 데이터를 쉼여두었다가 덩어리로 묶어 쓰는 방식
func BenchmarkBufferedWrite(b *testing.B) {
    bufferedFile := bufio.NewWriter(tmpFileOrFatal())
    performWrite(b, bufferedFile)
    bufferedFile.Flush()
}

func main() {
    resUnbuffered := testing.Benchmark(BenchmarkUnbufferedWrite)
    resBuffered := testing.Benchmark(BenchmarkBufferedWrite)

    fmt.Printf("BenchmarkUnbufferedWrite-8Wt%dWt%d ns/op\n", resUnbuffered.N,
        resUnbuffered.NsPerOp())
    fmt.Printf("BenchmarkBufferedWrite-8Wt%dWt%d ns/op\n", resBuffered.N,
        resBuffered.NsPerOp())
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

BenchmarkUnbufferedWrite-8	5000003969 ns/op
BenchmarkBufferedWrite-8	1000000 1356 ns/op

대기열 버퍼 큐를 활용하여 청킹 배치 연산을 수행했더니, 매번 하드웨어를 두들기던 무거운 비용이 사라지면서 속도가 무려 약 3배 가량 대폭 가속된 것을 볼 수 있다. 이처럼 데이터베이스 트랜잭션을 하나로 묶거나, 체크섬 대량 연산, 연속 메모리 공간 할당 등 “실행할 때마다 고정적인 오버헤드 비용이 무겁게 뒤따르는 작업”을 다룰 때는 큐를 통한 배치 처리가 시스템 전체 성능 향상에 거대한 정답이 된다.

(4) 시스템 전면 마비를 부르는 ‘죽음의 나선’과 해결책

대기열 큐가 발휘하는 가장 위대한 가치 중 하나는, 바로 대규모 분산 시스템 아키텍처 환경에서 시스템 전체가 연쇄적으로 터져서 영원히 복구 불가능한 상태로 가라앉는 ‘죽음의 나선(Death-Spiral, 이른바 부정적 피드백 루프)’ 현상을 기하학적으로 차단해 준다는 점이다.

서버의 처리 효율성이 특정 임계점 이하로 떨어져 응답이 아주 미세하게 지연되기 시작하면, 바깥에서 대기하던 클라이언트 앱이나 유저들은 “어? 왜 응답이 없지?”하면서 새로고침 버튼을 사정없이 연타하거나 타임아웃 재요청(Retry) 트래픽을 수만 배씩 더 강하게 서버로 던지기 시작한다.

안 그래도 과부하로 죽어가던 서버 파이프라인 마당에 분노한 유저들의 대량 재요청 폭탄 트래픽 2차로 쏟아져 들어오면, 서버는 더더욱 연산 불능 상태에 빠지고, 지연 시간은 기하급수적으로 늘어나며, 이에 질세라 클라이언트는 더 강한 재시도 폭탄을 던지는 악순환의 지옥문이 열린다. 안정장치가 없다면 시스템은 수동으로 재부팅하기 전까지 영원히 좀비 상태에서 회복되지 못한다.

이 파이프라인 진입로 맨 앞에 넉넉한 대기열 큐 창고를 뚫어놓으면 이 악순환의 고리를 물리적으로 끊어낼 수 있다. 들어오는 폭탄 트래픽들을 일단 안전한 메모리 대기열 창고에 전부 격리 수용해 두고, 서버 일꾼들은 외부 폭풍에 흔들리지 않은 채 창고에서 자기가 처리할 수 있는 정량의 일감만 묵묵히 꺼내와 연산에 집중하면 된다. 유저가 느끼는 반응 속도는 다소 느려질지언정, 최소한 서버 전체 아키텍처 시스템이 공멸하여 뺏어버리는 최악의 대참사는 완벽하게 방어해 낼 수 있다.

(5) 큐 크기 결정을 위한 리틀의 법칙(Little’s Law)

그렇다면 가장 중요한 질문이 있다.

“우리 파이프라인 시스템의 대기열 큐 크기(Buffer Size)는 대체 몇 칸으로 설정해야 가장 최적의 아키텍처가 될까?”

큐 알고리즘 학문에는 충분한 데이터 샘플링이 모였을 때 내 프로그램 파이프라인이 **수평 처리량**을 완벽하게 예측하고 결정해 주는 수학 공식이 존재한다.

이를 리틀의 법칙이라고 부르며, 공식은 다음과 같이 아주 심플하다.

$$L = \lambda W$$

- L : 현재 시스템 내부에서 연산 중이거나 큐에 대기하고 있는 전체 데이터(요청)의 평균 개수
- λ : 외부로부터 새로운 데이터(요청)가 유입되는 평균 도착율(Ingress Rate)
- W : 하나의 데이터(요청)가 시스템에 진입해서 완전히 처리되어 나갈 때 체류하는 총 평균 시간

이 공식은 상류에서 일감이 들어오는 속도와 하류에서 일을 마치고 나가는 속도가 균형을 이루는 시스템에서 완벽하게 성립한다. 만약 유입 속도가 나가는 속도를 초과하면 그 시스템은 언젠가 큐가 넘쳐 터지는 죽음의 나선 상태로 돌입하게 된다.

리틀의 법칙을 내 파이프라인 아키텍처 시스템에 대입해 보면 아주 흥미로운 사실을 수식으로 증명할 수 있다. 큐의 크기를 늘린다는 것은 공식에서 L 의 값을 인위적으로 증가시키는 행위이다. L 이 늘어나면 유입률(λ)이 동시에 꺾충 증가하거나, 혹은 하나의 요청이 시스템에 머무는 체류 시간(W)이 그만큼 길어지게 된다.

즉, 큐의 용량을 크게 키우면 키울수록 내 시스템의 자원 수용량은 넉넉해지지만, 유저가 체감하는 대기 지연 시간(Lag) 역시 그에 비례해서 무조건 엄청나게 길어진다는 수학적 진실이 성립한다.

큐 크기를 키우는 것은 공짜 점심이 아니라, ‘시스템 생존권’과 ‘지연 시간’을 서로 등가 교환하는 정교한 트레이드오프 연산이다.

실무 적용: 10만 건의 트래픽을 버티기 위한 적정 큐 크기 계산

내 파이프라인 시스템의 연산 스테이지가 총 3단계로 구성되어 있고, 샘플링 결과 요청 1건을 처리하는 데 평균 1밀리초($1\text{ms} = 0.001\text{s}$)가 소요된다고 가정한다.

이 하드웨어 스펙을 가지고 초당 무려 100,000건($100,000\text{ r/s}$)의 폭풍 트래픽이 밀려올 때 시스템이 터지지 않고 버티내기 위한 정석적인 대기열 큐 크기를 유도해 내는 수학적 연산 과정이다.

리틀의 법칙 공식의 L자리에서 현재 실제 CPU 연산 두뇌를 점유하고 일하는 3단계 스테이지 개수($3r$)를 차감해주면, 순수하게 필요한 대기열 창고의 크기가 도출된다.

$$L_r - 3r = 100,000\text{ r/s} \times 0.0001\text{ s}$$

$$L_r - 3r = 10$$

$$L_r = 13r$$

순수 필요한 큐의 크기 = $13_r - 3_r = 10\text{칸}$

수식 연산 결과, 우리 시스템이 이 대량의 트래픽 폭풍 속에서 죽음의 나선에 빠지지 않고 안정적인 방어벽을 유지하기 위해 필요한 적정 버퍼 채널의 크기는 정확히 ‘10칸’이라는 명쾌한 아키텍처적 정답 수치가 도출된다.

(6) 결론

대기열 큐의 상태별 핵심 운영 규칙을 요약하면 다음과 같다.

- 큐는 반드시 파이프라인의 맨 첫 진입으로 도어에 설치하거나, 혹은 청킹 배치 연산을 통해 극적인 효율이 보장되는 특정 가속 스테이지 공간에만 제한적으로 배치해야 한다.
- 연산 처리가 끝난 무거운 스테이지 뒤편에 습관적으로 버퍼 채널을 덕지덕지 바르는 행위는 시스템 처리량에 아무런 도움을 주지 못하며 자원낭 낭비하므로 철저히 피해야 한다.
- 하드웨어 장애나 예외 상황으로 인해 파이프라인에 패닉이 터지면 대기열 큐 메모리에 보관 중이던 모든 유저 요청 데이터가 흔적도 없이 증발하게 된다. 데이터 유실이 절대로 용납되지 않는 엄격한 금융성 환경이라면 인메모리 버퍼 채널 대신 디스크나 외부 저장소에 데이터를 먼저 안전하게 격리 보관하고 읽어오는 영구 보존형 영속 큐 아키텍처를 도입해야 한다.

큐잉 동시성 아키텍처 설계에서 가장 다이내믹하고 강력한 트래픽 제어 기술이지만, 시스템 전체의 데이터 흐름과 결합성으로 무겁게 변화시키므로, 항상 전체 정답 메커니즘을 먼저 완성해 둔 뒤 최후의 성능 튜닝 보루로 신중하게 도입하는 것이 맑고 깨끗한 코드를 유지하는 최고의 프로그래머 설계 방식이다.

[4-14] context 패키지

Go언어에서 비동기 및 멀티코어 연산의 생명 주기를 통제하고, 유기적으로 연결된 고루틴망을 안전하게 관리하는 최고의 무기인 context 패키지에 대해 깊이 있게 알아보겠다.

우리는 앞서 고루틴 누수를 방지하기 위해 부모가 done 채널을 개설하고 자식들이 이를 select 문으로 감시하는 방어벽 패턴을 배웠다.

이 방식은 대단히 훌륭하지만, 시스템의 규모가 커지면 한 가지 명확한 한계에 부딪히게 된다. 바로 “단순히 작업을 멈추라는 신호 외에, ‘왜 취소가 되었는지’ 혹은 ‘언제까지 일을 끝내야 하는지(데드라인)’같은 추가적인 컨텍스트 정보를 하류로 전달할 수 없다”는 점이다.

Go언어 개발진은 시스템의 크기와 상관없이 취소 신호와 함께 이 정보를 묶어서 전달하는 패턴이 필수적임을 깨달았다. 초기에는 표준 라이브러리 외부의 실험적 패키지였으나, Go 1.7버전부터 context 패키지 표준 라이브러리로 공식 편입되면서 현대 Go 동시성 프로그래밍에서 결코 빠질 수 없는 표준 이디엄(Standard idiom)으로 자리잡았다.

(1) context 패키지의 기본 구조와 인터페이스 스펙

context 패키지의 내부 스펙을 들여보다면 생각보다 아주 심플하고 직관적인 구조로 짜여 있다.

```
// 패키지가 제공하는 핵심 에러 및 함수 스펙 구조 개요
var Canceled = errors.New("context canceled")
var DeadlineExceeded error = deadlineExceededError{}

type CancelFunc func()
type Context interface {
    Deadline() (deadline time.Time, ok bool)
    Done() <-chan struct{}
    Err() error
    Value(key interface{}) interface{}
}
```

우리가 가장 주목해야 할 핵심은 바로 context.Context 인터페이스 타입이다. 이 타입은 기존의 done 채널처럼 내 프로그램의 전체 호출 스택 함수 흐름을 타고 하류로 자연스럽게 흘러내려간다.

Go언어의 강력한 표준 관례상, 동시성을 다루거나 네트워크 I/O를 수행하는 모든 하류 함수들은 반드시 ctx context.Context를 자신의 ‘첫 번째 매개변수’로 선언해야 한다. 인터페이스 내부 메서드들의 명확한 역할은 다음과 같다.

- Done()

기존의 done 채널과 완벽하게 동일한 역할을 수행한다. 부모가 작업을 취소하면 이 메시지가 반환하는 채널이 자동으로 닫히며(close), 하류 고루틴들은 이를 감시하듯 블로킹을 깨고 즉시 탈출할 수 있다.

- Deadline()

이 고루틴 작업이 언제 최종적으로 타임아웃되어 파괴될 예정인지 정확한 시각(time.Time)을 알려준다.

- Err()

Done() 채널이 닫힌 후, 이 작업이 도대체 ‘왜’ 취소되었는지 구체적인 원인 에러를 반환한다. 부모가 수동으로 취소했다면 Canceled 에러를, 시간이 다 되어 터졌다면 DeadlineExceeded 에러를 뱉어낸다.

- Value()

취소 신호 외에, 현재 요청과 밀접하게 연관된 핵심 데이터(예: 유저 ID, 인증 토큰 등)를 컨텍스트 주머니에 담아서 하류로 배달할 때 사용한다.

즉, context 패키지는 크게 두 가지 핵심 사명을 띠고 움직인다.

1. 호출 그래프 상의 모든 고루틴 자식 가지들을 안전하게 단 한 방에 취소시키는 API 제공
2. 요청 범위의 데이터를 전체 호출 스택 밑바닥까지 안전하게 실어 나르는 데이터 가방 역할 수행

먼저 첫 번째, 사명인 ‘취소(Cancellation)’ 제어 기법부터 상세히 보자.

(3) 고전적인 Done 채널 방식과의 아키텍처 비교

context 패키지가 가져다주는 구조적 축복을 눈으로 확인하기 위해, 먼저 우리가 이전 장에서 작성했던 고전적인 done 채널 기반의 다중 고루틴 조율 코드를 생략없는 전체 코드로 확인하자. 국가별 언어코드를 파악해서 인사말과 작별 인사를 동시에 출력하는 동시성 프로그램이다.

```
package main

import (
    "fmt"
    "sync"
    "time"
)

func main() {
    var wg sync.WaitGroup
    done := make(chan interface{})
    defer close(done)

    wg.Add(1)
    go func() {
        defer wg.Done()
        if err := printGreeting(done); err != nil {
            fmt.Printf("%v\n", err)
            return
        }
    }()

    wg.Add(1)
    go func() {
        defer wg.Done()
        if err := printFarewell(done); err != nil {
            fmt.Printf("%v\n", err)
        }
    }()

    wg.Wait()
}
```

```

        return
    }
}()

wg.Wait()
}

func printGreeting(done <-chan interface{}) error {
    greeting, err := genGreeting(done)
    if err != nil {
        return err
    }
    fmt.Printf("%s world!\n", greeting)
    return nil
}

func printFarewell(done <-chan interface{}) error {
    farewell, err := genFarewell(done)
    if err != nil {
        return err
    }
    fmt.Printf("%s world!\n", farewell)
    return nil
}

func genGreeting(done <-chan interface{}) (string, error) {
    switch locale, err := locale(done); {
    case err != nil:
        return "", err
    case locale == "EN/US":
        return "hello", nil
    }
    return "", fmt.Errorf("unsupported locale")
}

func genFarewell(done <-chan interface{}) (string, error) {
    switch locale, err := locale(done); {
    case err != nil:
        return "", err
    case locale == "EN/US":
        return "goodbye", nil
    }
    return "", fmt.Errorf("unsupported locale")
}

func locale(done <-chan interface{}) (string, error) {
    select {
    case <-done:
        return "", fmt.Errorf("canceled")
    case <-time.After(1 * time.Second): // 시뮬레이션을 위해 짧은 대기 설정
    }
    return "EN/US", nil
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
goodbye world!  
hello world!
```

이 고전적인 구조에서는 main 함수에서 close(done)을 외치면 프로그램의 양대 자식 산맥 고루틴들이 한 방에 죽는 제어는 훌륭하게 수행된다. 하지만 만약 비즈니스 요구사항이 다음과 같이 고차원으로 변경되면 이 done채널 방식은 소스코드가 끔찍하게 꼬이기 시작한다.

새로운 고급 동시성 연산 요구사항

1. 다른 파트는 괜찮으넌, 인사말을 생성하는 genGreeting 단계만큼은 무조건 딱1 초만 기다려보고 응답이 안오면 타임아웃으로 과감히 취소 탈출해라.
2. 시스템의 연대 책임 논리에 따라, 만약 인사말 출력(printGreeting)이 타임아웃 등으로 실패하면 작별 인사를 건네는 printFarewell 고루틴 수송관도 연쇄적으로 함께 취소시켜 버려라. (인사도 안해놓고 작별인사를 나누는 것은 논리에 맞지 않다.)

이 정교한 취소 체인 트리를 context 패키지로 전환하여 완벽하게 구현한 정석적인 전체 코드를 확인해보자.

```
package main  
  
import (  
    "context"  
    "fmt"  
    "sync"  
    "time"  
)  
  
func main() {  
    var wg sync.WaitGroup  
  
    // 1. 뿌리가 되는 최상위 컨텍스트를 WithCancel로 감싸 수동 취소 권한을 확보함  
    ctx, cancel := context.WithCancel(context.Background())  
    defer cancel()  
  
    wg.Add(1)  
    go func() {  
        defer wg.Done()  
        if err := printGreeting(ctx); err != nil {  
            fmt.Printf("cannot print greeting: %v\n", err)  
            // 연대 책임 발동: 인사말 출력이 실패하면 부모의 취소 버튼을 눌러  
            // 반대편 진영에 살아있던 작별 인사 고루틴 트리까지 통째로 연쇄 퇴근  
            시킵니다.  
            cancel()  
        }  
    }()  
  
    wg.Add(1)  
    go func() {  
        defer wg.Done()
```

```

        if err := printFarewell(ctx); err != nil {
            fmt.Printf("cannot print farewell: %v\n", err)
        }
    }0

    wg.Wait()
}

func printGreeting(ctx context.Context) error {
    greeting, err := genGreeting(ctx)
    if err != nil {
        return err
    }
    fmt.Printf("%s world!\n", greeting)
    return nil
}

func printFarewell(ctx context.Context) error {
    farewell, err := genFarewell(ctx)
    if err != nil {
        return err
    }
    fmt.Printf("%s world!\n", farewell)
    return nil
}

func genGreeting(ctx context.Context) (string, error) {
    // 독자 노선 구축: 부모의 컨텍스트 철학은 유지한 채, 자신만을 위한 1초짜리 타임아웃
    // 자식 방어벽을 새로 조립함
    ctx, cancel := context.WithTimeout(ctx, 1*time.Second)
    defer cancel()

    switch locale, err := locale(ctx); {
    case err != nil:
        return "", err
    case locale == "EN/US":
        return "hello", nil
    }
    return "", fmt.Errorf("unsupported locale")
}

func genFarewell(ctx context.Context) (string, error) {
    switch locale, err := locale(ctx); {
    case err != nil:
        return "", err
    case locale == "EN/US":
        return "goodbye", nil
    }
    return "", fmt.Errorf("unsupported locale")
}

func locale(ctx context.Context) (string, error) {

```

```

select {
    case <-ctx.Done(): // 원인 추적: 이제 단순 멈춤이 아니라 부모가 뱉어내는 명확한 에러
        원인을 상류로 넘겨줍니다.
        return "", ctx.Err()
    case <-time.After(1 * time.Minute): // 의도적으로 1분간 대기하여 타임아웃 병목 확정
        유발
}
return "EN/US", nil
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

cannot print greeting: context deadline exceeded
cannot print farewell: context canceled

```

출력 지표 결과를 확인해보자.

아키텍처 연산이 우리가 설계한 논리 그대로 완벽하게 동작했다.

locale 함수가 1분 동안 버티며 서 있자, 1초짜리 타임아웃 바운더리를 쳐두었던 genGreeting 자식 스택이 먼저 타임아웃을 감지하고 무너지며 상류로 “context deadline exceeded”라는 정교한 고유 에러 사유를 배달했다.

이 실패 소식을 접한 메인 함수의 왼쪽 고루틴이 즉시 전체 취소(cancel())을 때려버렸고, 그 신호가 도미노처럼 하류로 흘러내려가 대기 중이던 오른쪽 작별 인사 진영까지 “context canceled”라는 에러와 함께 낙오자 없이 완벽하게 진압 완료된 것이다. 관심사의 간섭 없이 코드의 유연한 결합성만으로 대형 조율 시스템을 완성해 냈다.

(4) 고속 실패 처리를 위한 데드락 사전 검증 기법

여기서 한 차원 더 높은 최적화 아키텍처 기법을 녹여낼 수 있다.

locale 함수 관점에서 가만히 보니까 “어차피 내 연산을 끝마치는데 평균 1분 소요되는데, 방금 부모가 넘겨준 컨텍스트 가방의 타임아웃 만료 시계를 확인해 보니 고작 1초밖에 안남았네? 어차피 시간 내에 골인하는게 물리적으로 불가능하잖아!”라는 상황을 미리 눈치챌 수 있다.

어차피 실패할 연산이라면 무의미하게 1초 동안 가만히 누워서 시간을 버리다가 터지는 것보다, 진입하자마자 시계를 대조해 보고 즉시 연산을 거절하는 고속 실패(Fail-Fast) 구조를 만드는 것이 시스템 자원 관리 상 훨씬 이롭다.

Context.Deadline() 메서드를 결합한 고속 실패 레이아웃 전체 코드를 확인해보자.

```

package main

import (
    "context"
    "fmt"
    "sync"
    "time"
)

func main() {
    var wg sync.WaitGroup

```

```

ctx, cancel := context.WithCancel(context.Background())
defer cancel()

wg.Add(1)
go func() {
    defer wg.Done()
    if err := printGreeting(ctx); err != nil {
        fmt.Printf("cannot print greeting: %v\n", err)
        cancel()
    }
}()

wg.Add(1)
go func() {
    defer wg.Done()
    if err := printFarewell(ctx); err != nil {
        fmt.Printf("cannot print farewell: %v\n", err)
    }
}()

wg.Wait()
}

func printGreeting(ctx context.Context) error {
    greeting, err := genGreeting(ctx)
    if err != nil {
        return err
    }
    fmt.Printf("%s world!\n", greeting)
    return nil
}

func printFarewell(ctx context.Context) error {
    farewell, err := genFarewell(ctx)
    if err != nil {
        return err
    }
    fmt.Printf("%s world!\n", farewell)
    return nil
}

func genGreeting(ctx context.Context) (string, error) {
    ctx, cancel := context.WithTimeout(ctx, 1*time.Second)
    defer cancel()

    switch locale, err := locale(ctx); {
    case err != nil:
        return "", err
    case locale == "EN/US":
        return "hello", nil
    }
    return "", fmt.Errorf("unsupported locale")
}

```



```

}

func genFarewell(ctx context.Context) (string, error) {
    switch locale, err := locale(ctx); {
        case err != nil:
            return "", err
        case locale == "EN/US":
            return "goodbye", nil
        }
    return "", fmt.Errorf("unsupported locale")
}

func locale(ctx context.Context) (string, error) {
    // 고속 실패 최적화 마스터 피스: 진입 즉시 부모가 설정한 만료 시각을 샘플링하여 대조
    함
    if deadline, ok := ctx.Deadline(); ok {
        // 만약 지금 내 연산 소요 시간(1분)을 더한 시각이 약속된 만료 기한을 초과해
        버린다면
        if deadline.Sub(time.Now().Add(1*time.Minute)) <= 0 {
            // 1초간 미련하게 sleep 대기하지 않고 그 즉시 표준 라이브러리의
            DeadlineExceeded 표준 에러를 뱉으며 즉시 즉시 탈출합니다.
            return "", context.DeadlineExceeded
        }
    }

    select {
        case <-ctx.Done():
            return "", ctx.Err()
        case <-time.After(1 * time.Minute):
    }
    return "EN/US", nil
}

```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```

cannot print greeting: context deadline exceeded
cannot print farewell: context canceled

```

출력 결과물 지표 자체는 아까와 동일하지만, 내부적인 리소스 점유 타임라인은 완전히 달라졌다. locale 함수가 고속 실패 방어벽 덕분에 단 1나노초 만에 연산을 거부하고 튀어나왔기 때문에 컴퓨터 두뇌는 쓸데없는 지연 소모 시간 없이 최고 속도로 다음 멀티코어 업무 배정 흐름으로 안전하게 넘어갈 수 있게 된 것이다.

(5) 컨텍스트 가방을 통한 요청 범위 데이터 유동

이제 context 패키지가 가진 두 번째 위대한 사명인 ‘요청 범위의 데이터 실어나르기(Data-bag)’ 메커니즘을 정복해보자.

웹 서비스 환경에서 고루틴 일꾼이 하나 생성되어 유저의 요청을 처리하기 시작하면, 그 고루틴 스택 깊숙한 골짜기에 박혀있는 저수준 데이터베이스 핸들러나 암호화 함수들까지 현재 요청을 보낸 유저의 식별 정보가 연속적으로 배달되어야 할 때가 많다. 이를 위해 사용하는 팩토리 함수가 바로 context.Withvalue이다.

```
package main

import (
    "context"
    "fmt"
)

func main() {
    // 메인 진입로에서 유저 userID 정보와 인증 토큰 문자열을 주입함
    ProcessRequest("jane", "abc123")
}

func ProcessRequest(userID, authToken string) {
    // 빈 부모 컨텍스트 주머니 속에 키-값 쌍으로 데이터를 적재하며 가방을 확장해 나감
    ctx := context.WithValue(context.Background(), "userID", userID)
    ctx = context.WithValue(ctx, "authToken", authToken)

    HandleResponse(ctx)
}

func HandleResponse(ctx context.Context) {
    // 호출 스택 최하단 함수는 가방에서 키 값을 밀어 넣어 원하는 자원을 끄집어내 사용합니다.
    fmt.Printf("handling response for %v (%v)\n", ctx.Value("userID"),
        ctx.Value("authToken"))
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
handling response for jane (abc123)
```

interface{} 기반 데이터 가방이 가진 구조적 위험성과 우회 기법

WithValue 함수는 키와 값의 타입을 모두 최고 조상 타입인 interface{} 형태로 받아들인다. 이 말은 즉, Go 언어가 자랑하는 든든한 ‘컴파일 타임의 타입 안정성’의 보호막이 완전히 해제된다는 무서운 뜻이다.

다른 외부 서드파티 패키지 라이브러리 개발자가 우연히 똑같이 “userID”라는 문자열을 자기 패키지의 컨텍스트 키 값으로 사용해버리면, 내 주머니 속 데이터가 엉뚱하게 오염되어 버리는 치명적인 키 충돌 대참사가 일어난다. 또한 값을 꺼낼 때 타입 캐스팅.(string))을 개발자가 일일이 수동으로 수행해야 하므로 런타임 캐스팅 패닉 에러 위험성에도 상시 노출된다.

이 키 충돌 문제를 완벽하게 회복하고 우회하는 Go 엔지니어들의 마스터 설계 규칙이 있다.

패키지 내부에 외부로 절대 수출하지 않는(Private) 패키지 전용 고유 키 타입을 선언하고, 데이터를 주입하고 추출하는 행위 자체를 타입 안정성이 보장된 정적 전용 헬퍼 함수로 캡슐화하여 수출하는 방식이다. 정석 구조의 전체 코드를 확인해보자.

```
package main

import (
    "context"
    "fmt"
)

// 규칙 1: 외부 패키지에서 절대 쳐다볼 수 없도록 캡데기 전용 비공개 키 타입을 선언합니다.
type ctxKey int

// 규칙 2: iota 문맥을 활용해 고유한 내부 정수형 키 상수들을 개설합니다.
const (
    ctxUserID ctxKey = iota
    ctxAuthToken
)

// 규칙 3: 값을 추출하는 행위를 정적 타입이 강제된 안전한 전용 함수(Accessor)로 감싸서 수출
// 합니다.
func UserID(c context.Context) string {
    return c.Value(ctxUserID).(string)
}

func AuthToken(c context.Context) string {
    return c.Value(ctxAuthToken).(string)
}

func main() {
    ProcessRequest("jane", "abc123")
}

func ProcessRequest(userID, authToken string) {
    // 데이터를 적재할 때 비공개 엄격 타입 키인 ctxUserID, ctxAuthToken 상수를 사용하
    // 므로
    // 외부 그 어떤 패키지도 이 주머니 자리를 침범하거나 오염시킬 수 없습니다.
    ctx := context.WithValue(context.Background(), ctxUserID, userID)
    ctx = context.WithValue(ctx, ctxAuthToken, authToken)

    HandleResponse(ctx)
}

func HandleResponse(ctx context.Context) {
    // 소비 측 유저는 내부 키가 정수형인지 문자열인지 신경 쓸 필요 없이,
    // 타입 안정성이 100% 보장된 헬퍼 함수를 통해 안전하게 자원을 수집합니다.
    fmt.Printf("handling response for %v (auth: %v)\n", UserID(ctx), AuthToken(ctx))
}
```

위 코드를 실행하면 다음과 같은 결과가 출력된다.

```
handling response for jane (auth: abc123)
```

(6) 컨텍스트 가방에 담을 올바른 데이터 판별 법칙

context 패키지의 데이터 가방 기능은 유용함과 동시에 Go 커뮤니티 내에서 꽤 많은 논쟁을 일으키는 주제이다. 아무 데이터나 다 담아도 되는 무법지대로 변질되기 쉽기 때문이다. Go 공식 문서에 이에 대해 다소 모호하지만 뼈가 있는 한 줄짜리 경고 주석을 달아두었다.

Go 개발진의 엄격한 경고

“컨텍스트 값(WithContext)은 오직 프로세스와 API 경계를 넘나드는 ‘요청 범위의 데이터’를 전달하는 용도로만 사용해야지, 함수에게 슬쩍 보낼 ‘선택적 매개변수’를 채워 넣는 편법 통로로 남용해서는 절대 안됩니다.”

무엇이 선택적 매개변수이고 무엇이 정당한 요청 범위 데이터인지 헷갈릴 때는, 저자가 제시하는 5가지 정밀 판별 가이드라인 휴리스틱 법칙에 대조해 보면 명쾌한 설계 정답을 내릴 수 있다.

판별 기준 내용 (Heuristics Guide)	요청 ID (Request ID)	유저 ID (User ID)	현재 요청 URL	API 서버 커 넥션 객체	권한 인증 토큰 (Auth Token)
1) 프로세스 및 API 경 계를 실제로 넘나드는 가?	✓	✓	✓	—	✓
2) 한 번 주입되면 변하 지 않는 불변 (Immutable)인가?	✓	✓	✓	—	✓
3) 외부 패키지 수용이 쉬운 단순형(Simple Type)인가?	✓	✓	✓	—	✓
4) 메서드가 없는 순수 한 데이터(Data) 구조인 가?	✓	✓	—	—	✓
5) 알고리즘 분기를 조 종하지 않고 데코레이션 만 하는가?	✓	✓	—	—	—
최종 컨텍스트 적재 적 합 여부 판단 결과	적합 (YES)	적합 (YES)	애매함	절대 불가 (NO)	적합 (YES)

데이터베이스의 실제 네트워크 연결 커넥션 객체(sql.DB 인스턴스 등)처럼 메서드들이 덕지덕지 붙어있고 연산 로직을 직접 수행하는 무거운 인프라 자원들은 절대로 컨텍스트 가방에 집어넣어서는 안된다.

그것은 투명하게 보여야 할 함수의 명시적 의존성 스펙을 감추어 프로그램 아키텍처를 추적 불가능한 스파게티 높이로 빠트리느 지름길이다.

반면 요청 ID나 인증 토큰처럼 연산의 성격을 보조하고 로그에 찍어줄 순수 데이터 값들은 컨텍스트에 담아 유통하는 것이 아주 훌륭한 설계이다.

(7) 결론

인터페이스 기반의 데이터 가방 영역에 다소 아쉬운 불호 논쟁이 있을지 언정, context 패키지가 제공하는 거대한 고루틴 취소 트리 제어 능력만큼은 멀티코어 분산 시스템을 구축할 때 컴퓨터 과학계가 제공하는 가장 축복에 가까운 핵심 도구임이 자명하다.

데이터 가방에 대한 미세한 거부감 때문에 이 패키지 도입으 주저하지 마시고, 든든한 `ctx.Done()`과 `context.WithTimeout` 무기를 장착하여 대규모 트래픽 폭풍 속에서도 고루틴 누수 없이 맑고 안전하게 살아 숨쉬는 우아한 동시성 아키텍처를 완성해보자.